The story of a transaction

From its birth to death and after-life

@stratospher on 17/5/2025

Index

1. What is a bitcoin transaction?
2. How does it look like?
3. Design goals
4. Evolution of bitcoin transactions
    - Legacy
    - Segwit
    - Taproot
5. Key takeaways

menti.com

enter code: 8275 3573



Transaction basics

Transaction

Move some “amount” from 1 place to another



INPUTS



OUTPUTS





How can you code this?



20

Transaction



20

Coins destroyed

Coins created

Transaction

Move some “amount” from 1 place to another



INPUTS





OUTPUTS

60 INR







UTXO = bitcoins you can spend

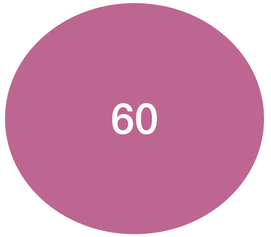
Transaction outputs that have not been consumed yet and can be used for spending





Transaction

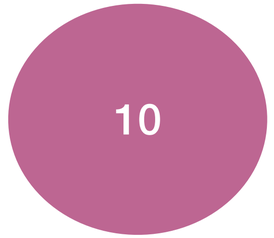




UTXO destroyed

UTXO created





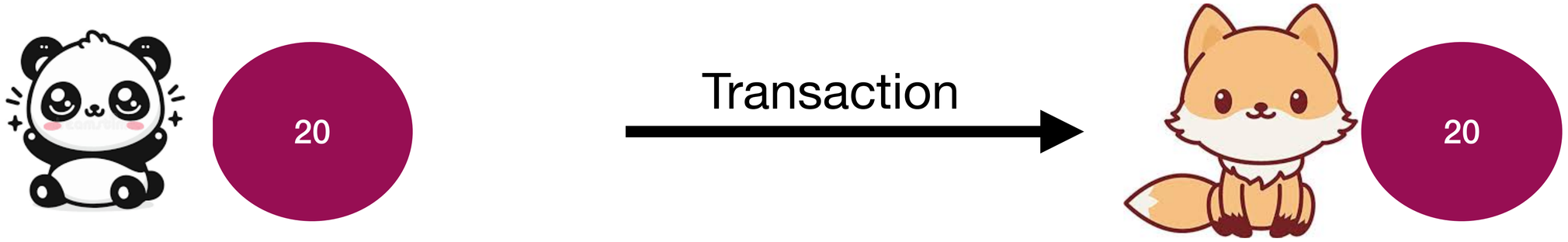
How to code?

Transaction effects

- Amount
- Source
- Destination

Permission?

- Signatures - I have permission to move the coins

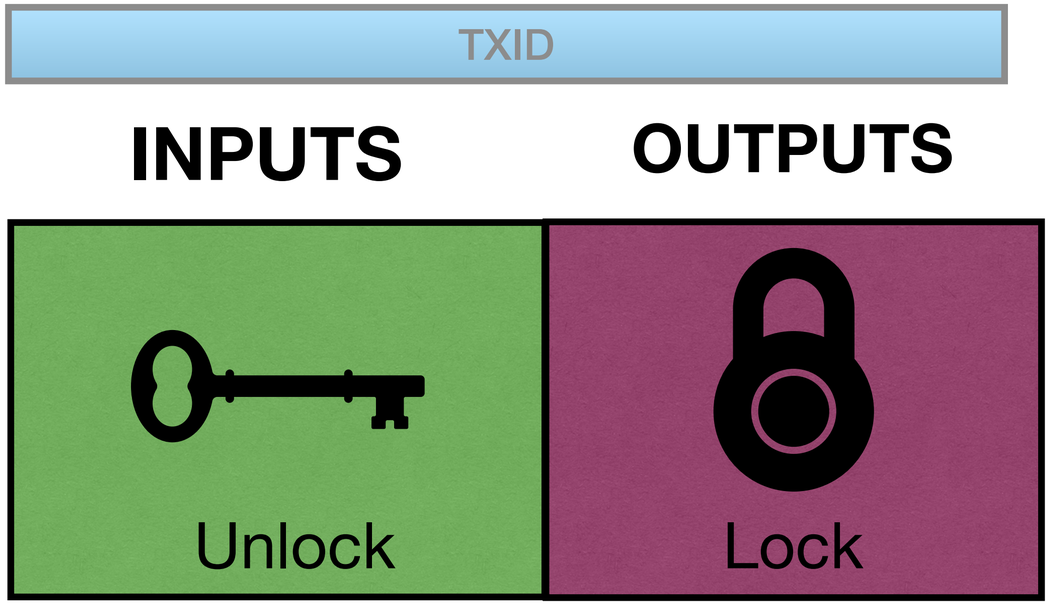


Coins destroyed

Coins created

Permission to make transaction?

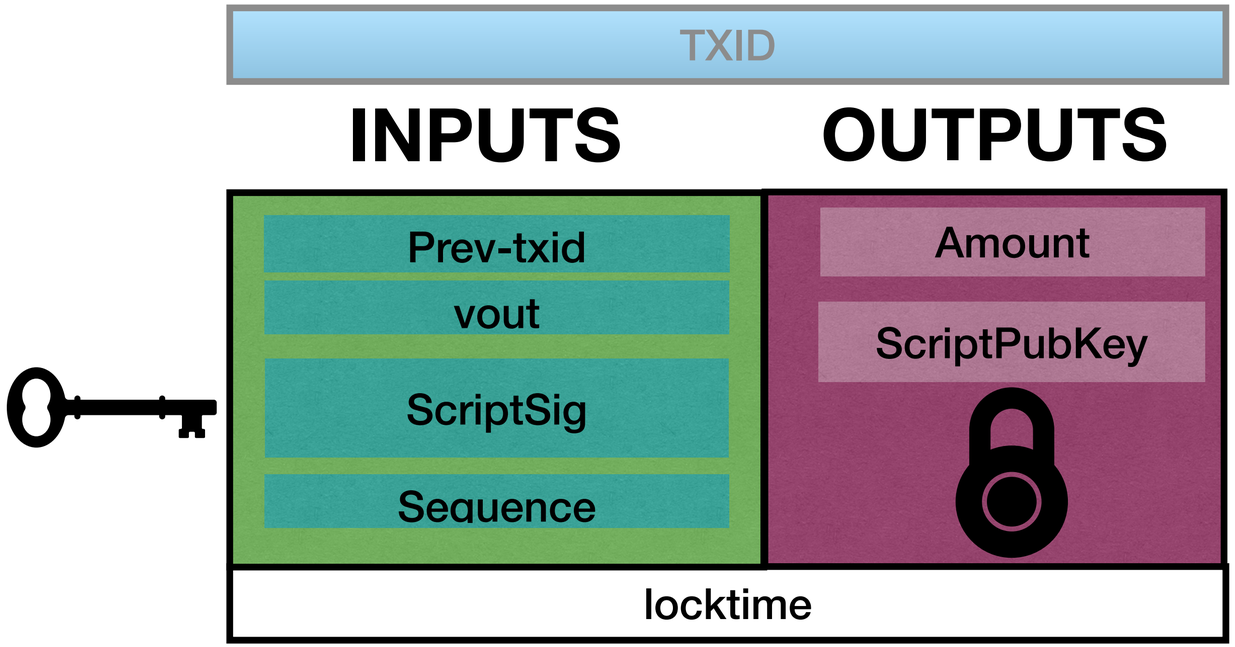
Permission = Bunch of data that unlocks and locks up batches of bitcoin



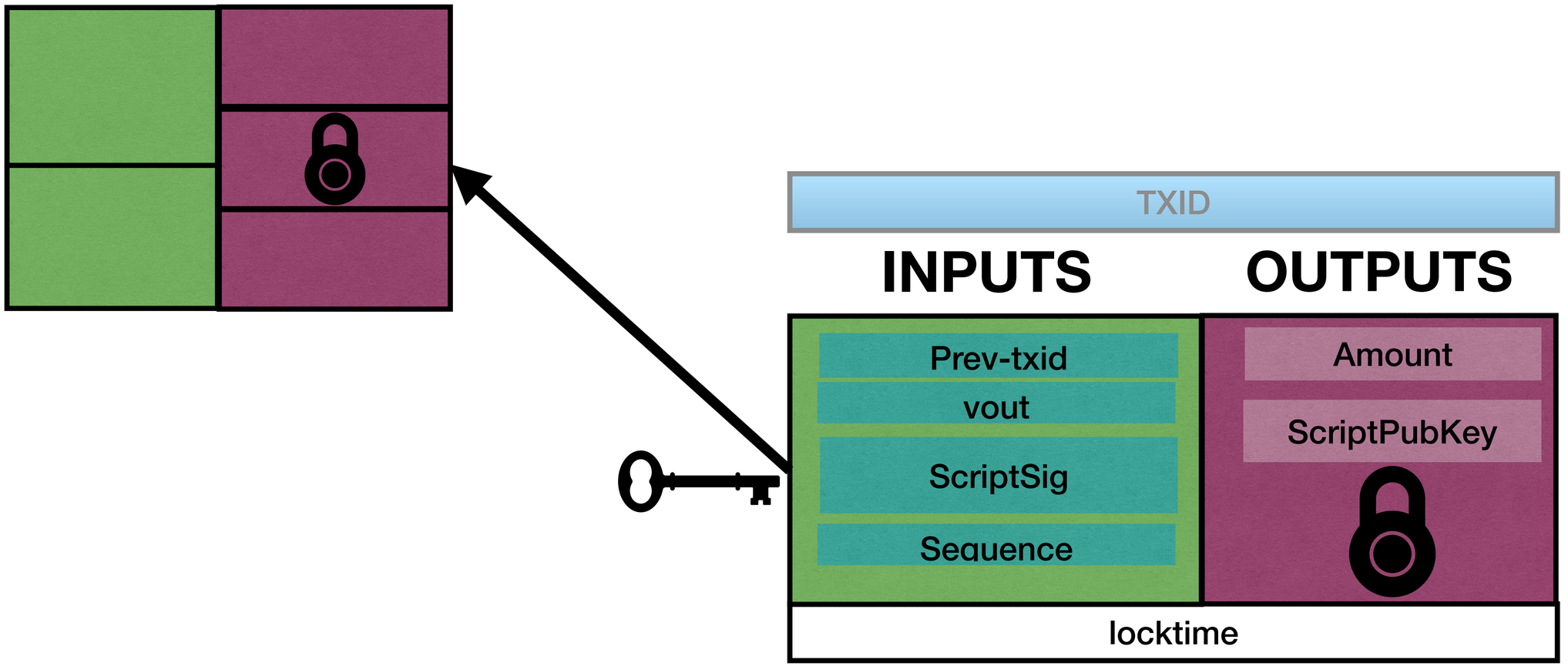
Scripts and Signatures

How does a transaction look?

Transaction

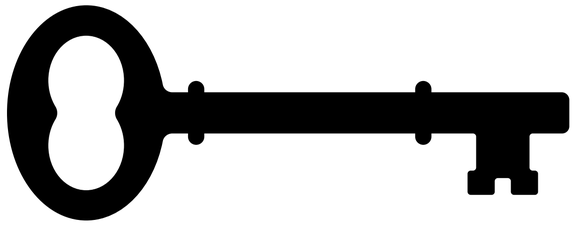


Prev-txid and vout

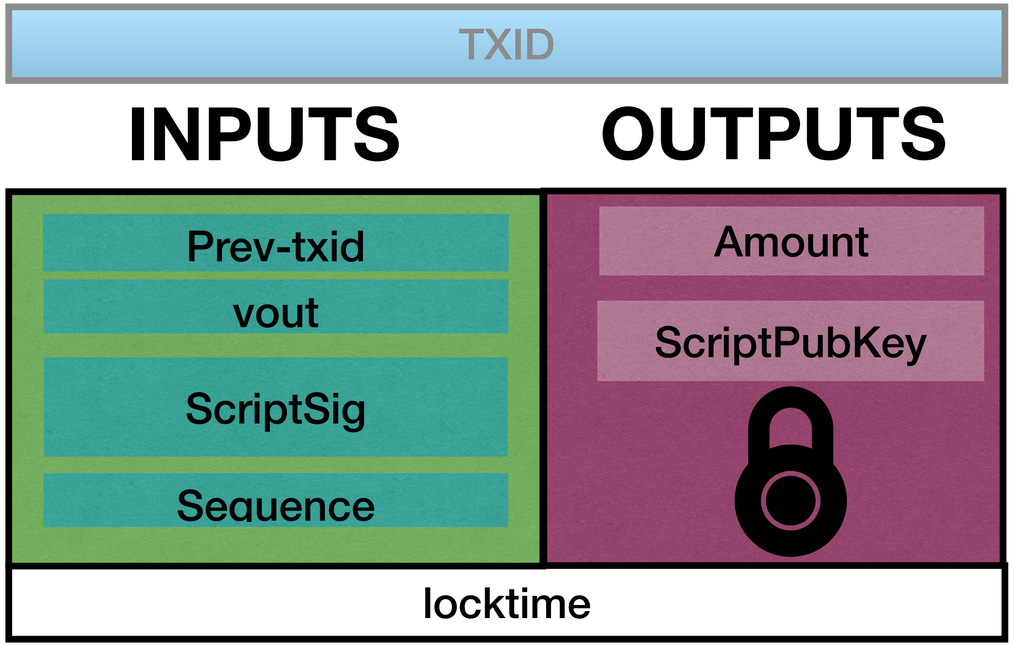


ScriptSig

Unlocking script

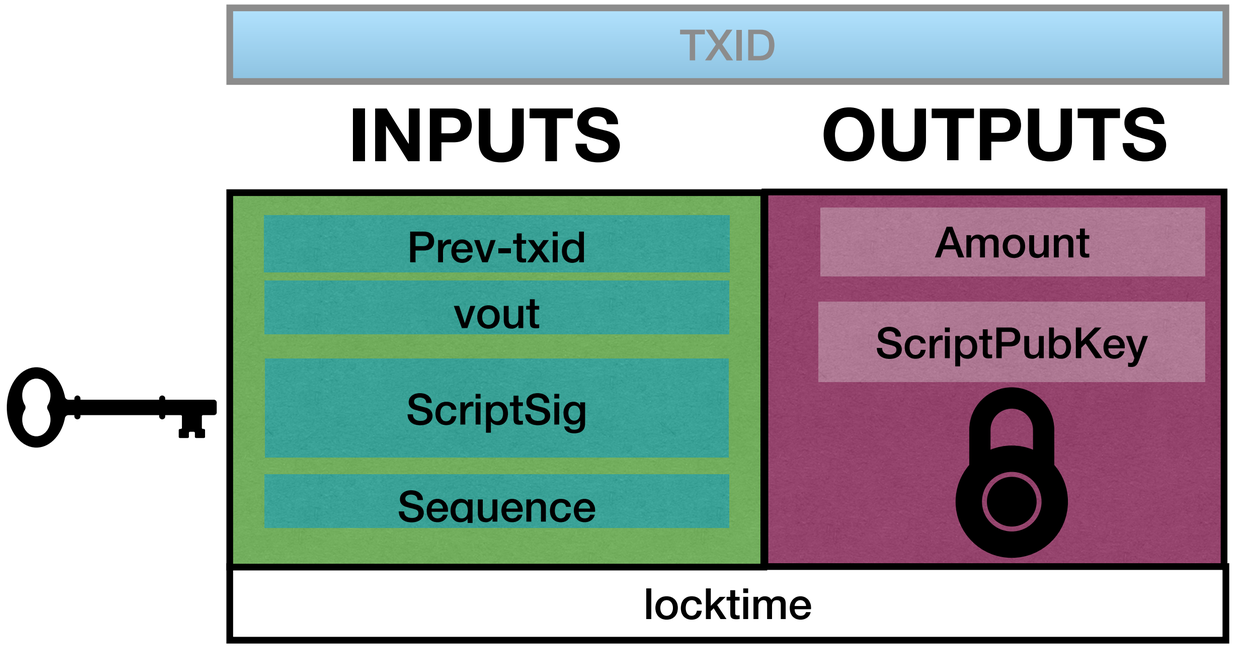


Signatures etc..



Sequence

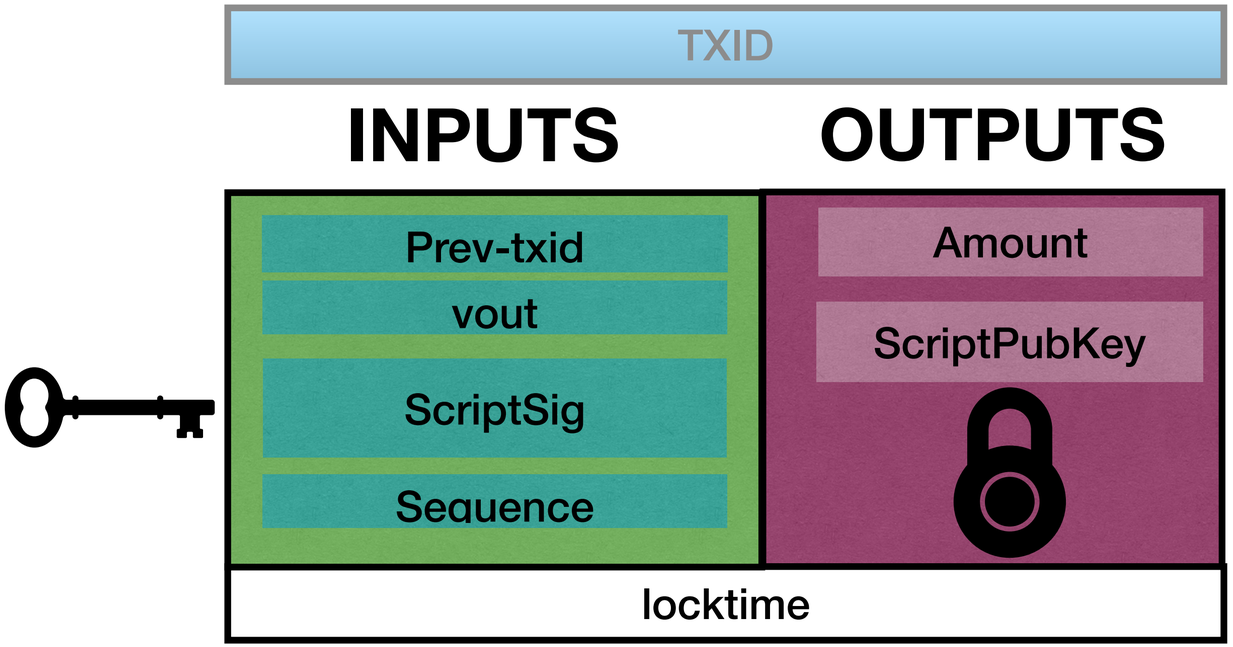
More advanced tweaking = when tx can be mined, can tx be replaced?



Amount

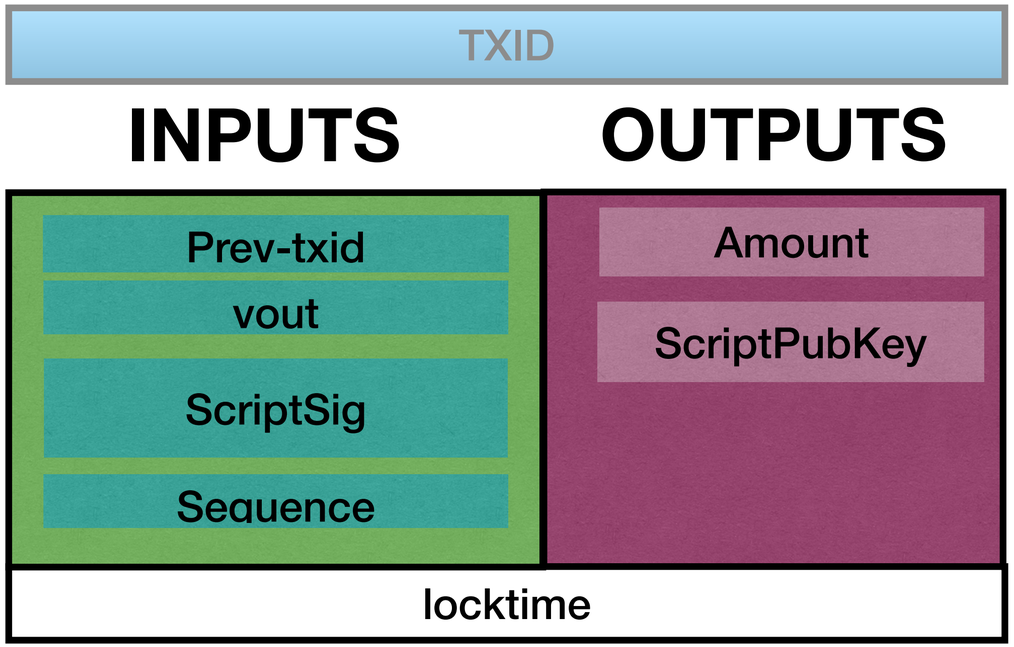
8 byte field ~ (theoretical) 184,467 million BTC





ScriptPubKey

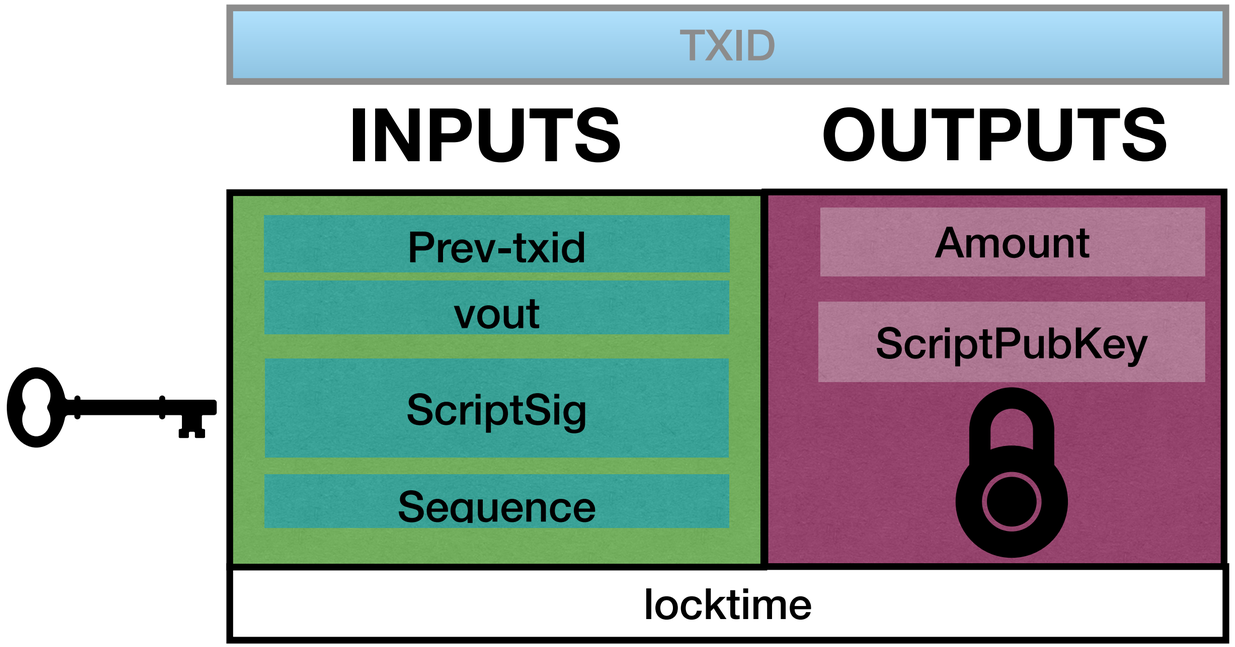
Locking script





locktime

prevent tx from being mined until after [block height/time]



Design goals



Privacy (don’t reveal too much user info)

Scalability (block space is limited)

Immutability (no one tampers)

Security (only rightful owner spends)

Evolution of bitcoin transactions

1. Legacy transaction

P2PK

P2PKH

P2SH

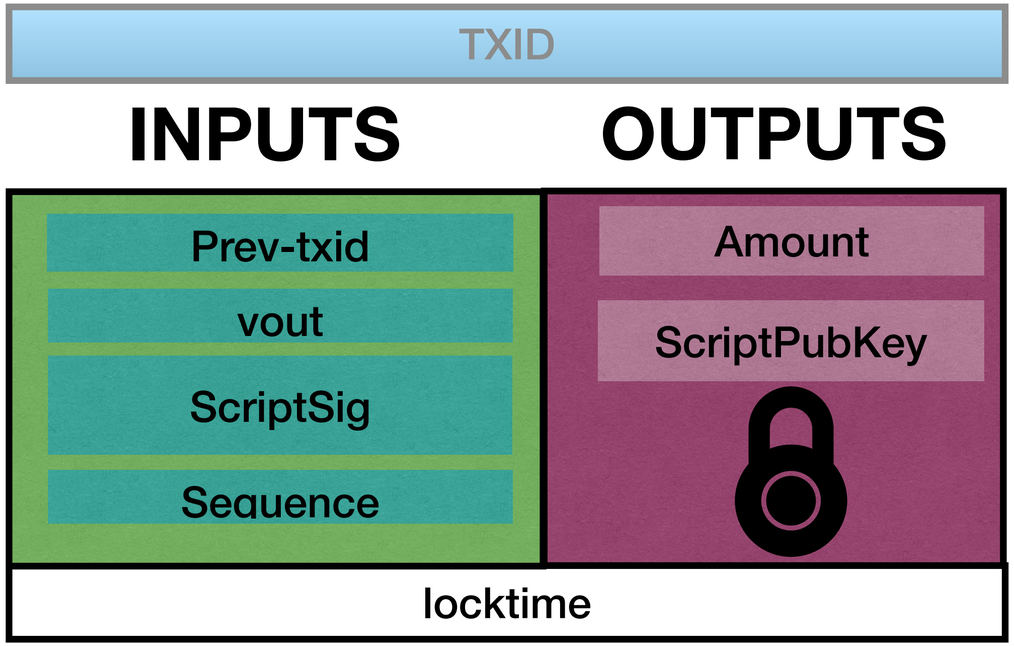
1. Legacy transaction

P2PK

legacy P2PK

P2PK

ScriptPubKey = locking script



ScriptPubKey

<pubkey> OP_CHECKSIG

legacy P2PK

Ex: Genesis block's locking script (P2PK)

ScriptPubKey

ASM | Hex

Public Key

```
OP_PUSHBYTES_65
🔑 04946420595018 8c29d377eebca6535e0f3699ce4069ecd77ffebfbd0bcf95e3c134cb7d2742d800a12df41413a09ef87a80516353a2f0a2
80547bb5512dc03da8
OP_CHECKSIG
```

Transaction: 1db6251a9afce7025a2061a19e63c700dffc3bec368bd1883decfac353357a9d (Output 1)

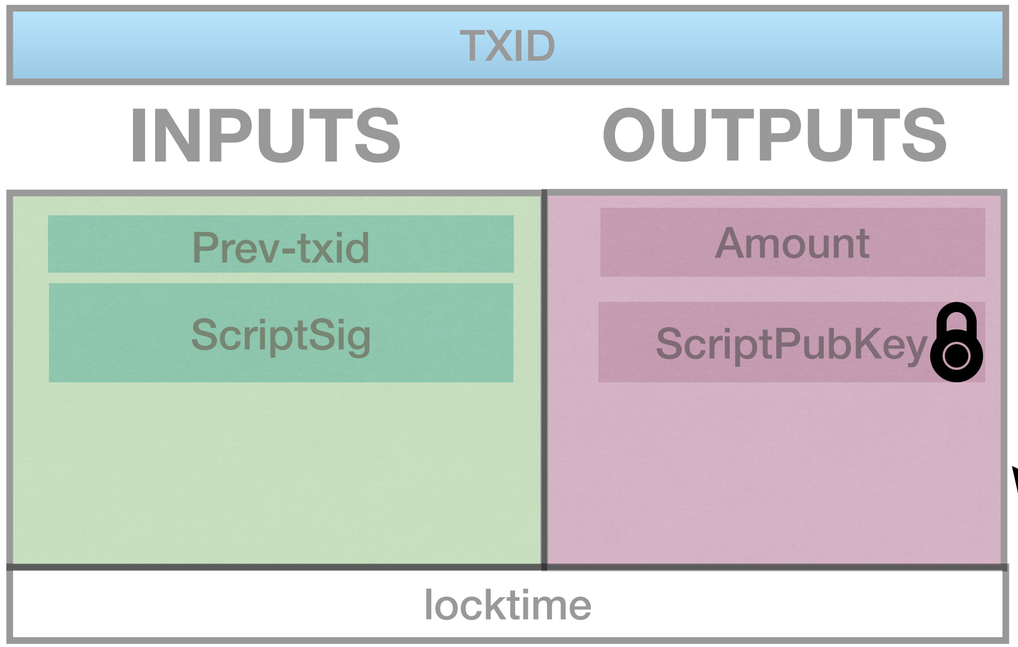
This example contains a 65-byte uncompressed 🔑 public key.

Tip: The 🔑 public key can either be an uncompressed public key (65 bytes) or a compressed public key (33 bytes). You can use either within a P2PK.

P2PK

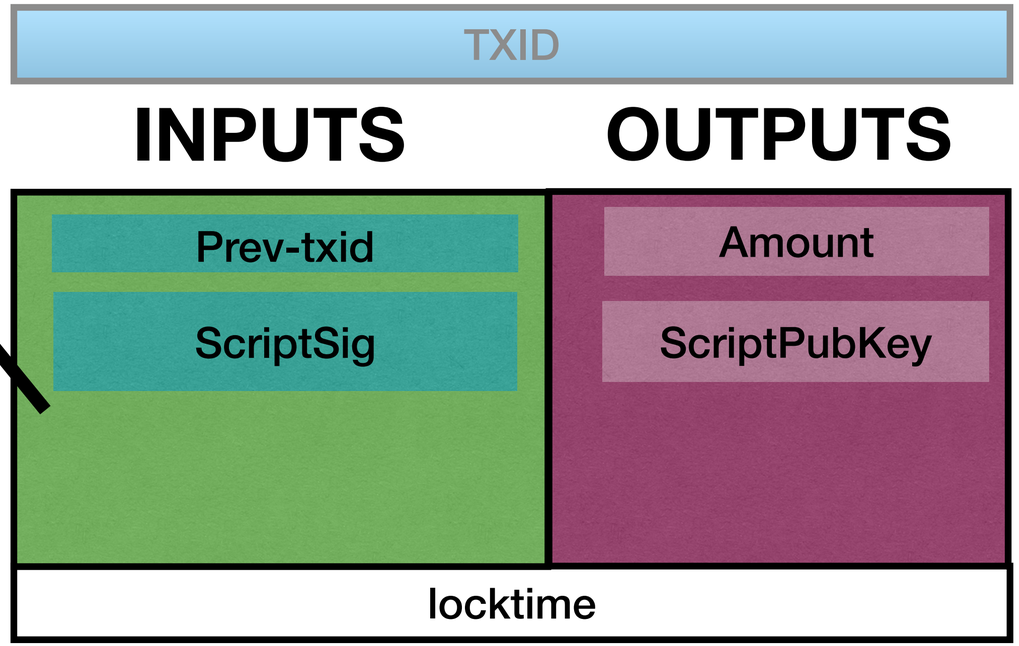
legacy P2PK

ScriptSig = unlocking script



ScriptSig





legacy P2PK

Successful unlocking of previous utxo?

If signature verification passes



scriptSig **scriptPubKey**

legacy P2PKH

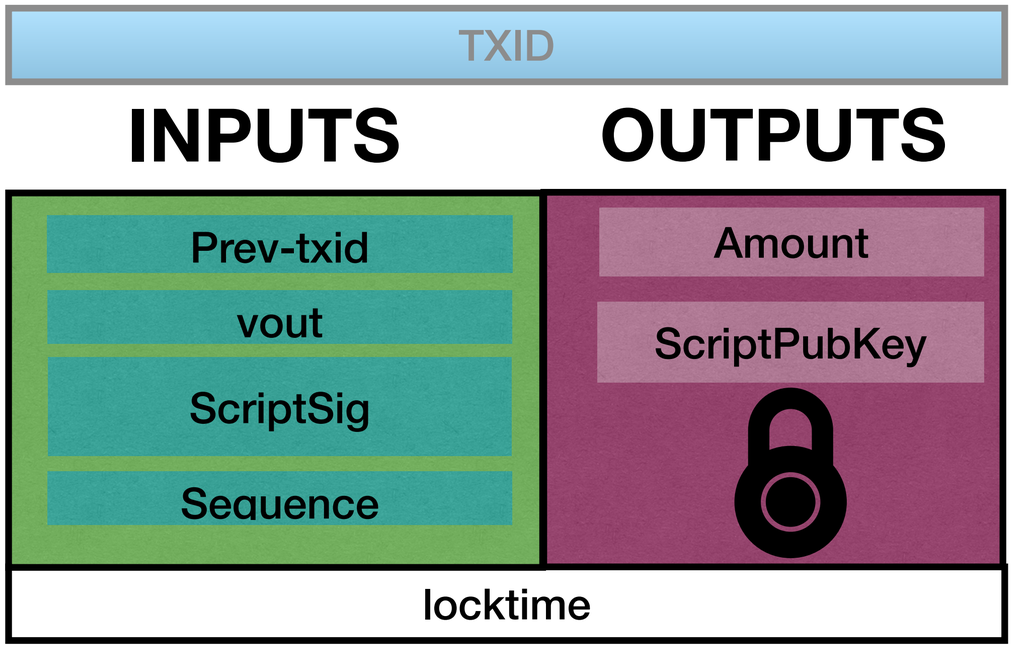
1. Legacy transaction

P2PKH

legacy P2PKH

P2PKH

ScriptPubKey = locking script



ScriptPubKey

OP_DUP OP_HASH160 <pubkey_hash> OP_CHECKSIG

legacy P2PKH

P2PKH locking script

ScriptPubKey

ASM | Hex

```
OP_DUP
OP_HASH160
OP_PUSHBYTES_
55ae51684c43435da751ac8d2173b2652eb64105
OP_EQUALVERIFY
OP_CHECKSIG
```

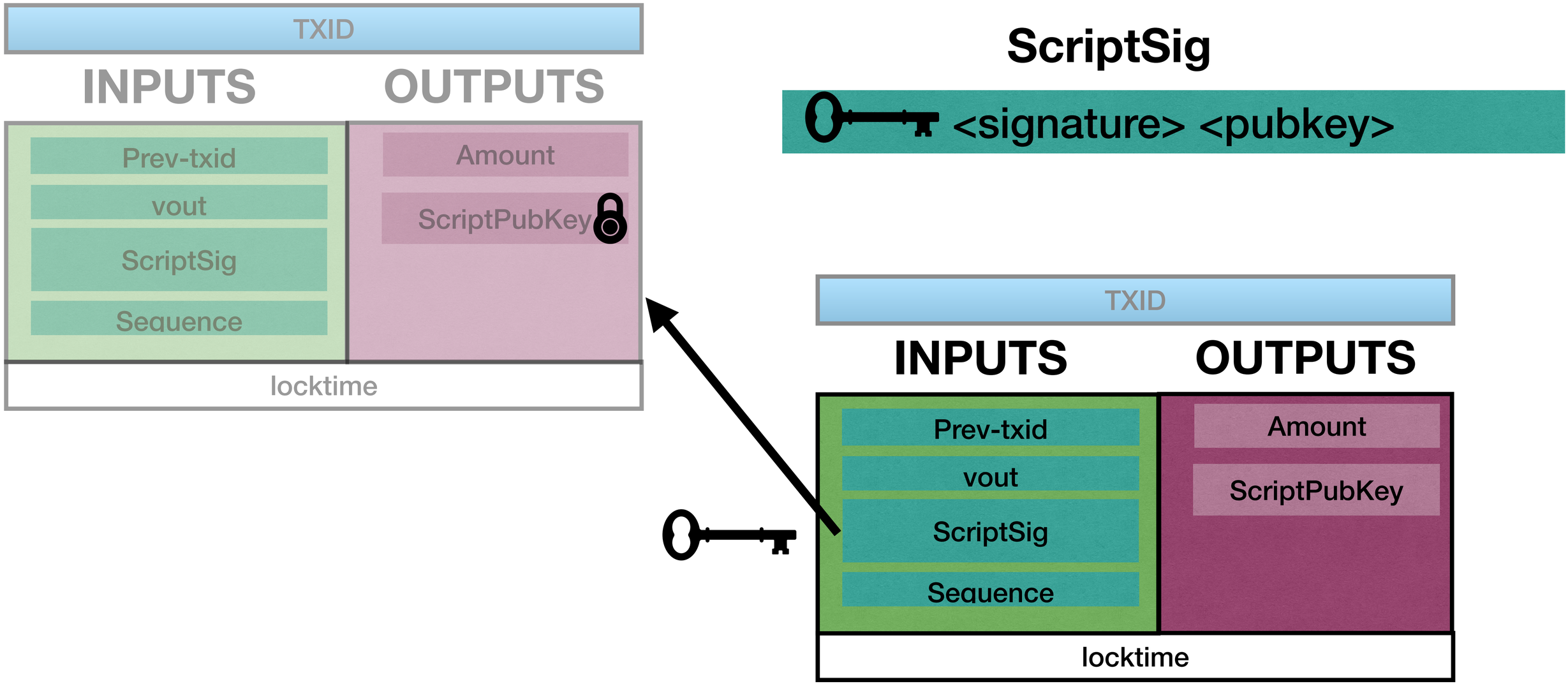
Public Key Hash

Transaction: 0b6461de422c46a221db99608fcbe0326e4f2325ebf2a47c9faf660ed61ee6a4 (Output 1)

P2PKH

legacy P2PKH

ScriptSig = unlocking script



P2PKH

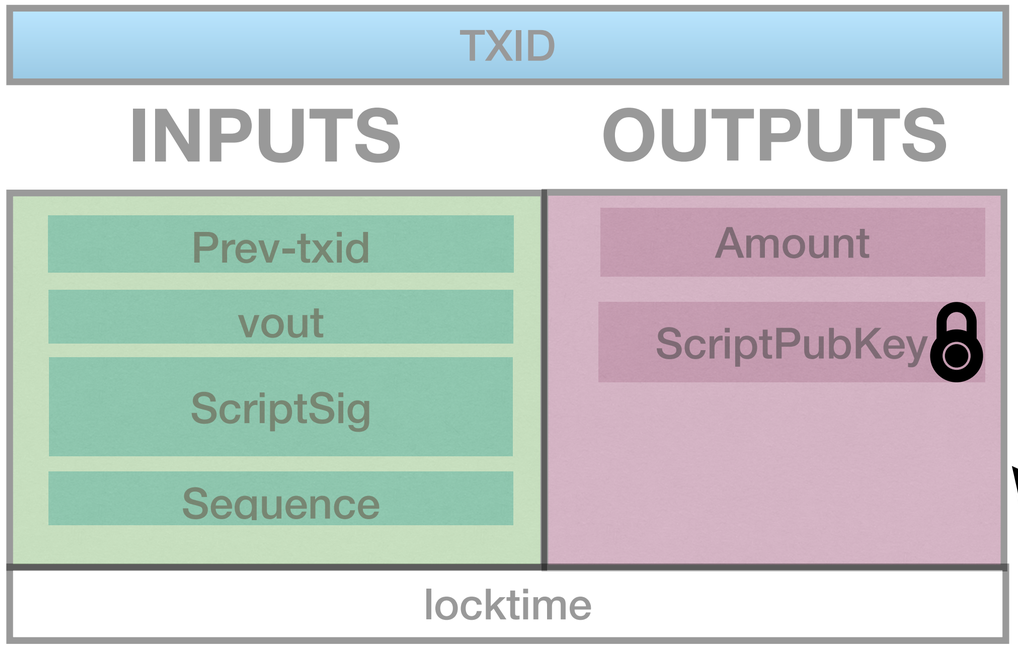
legacy P2PKH

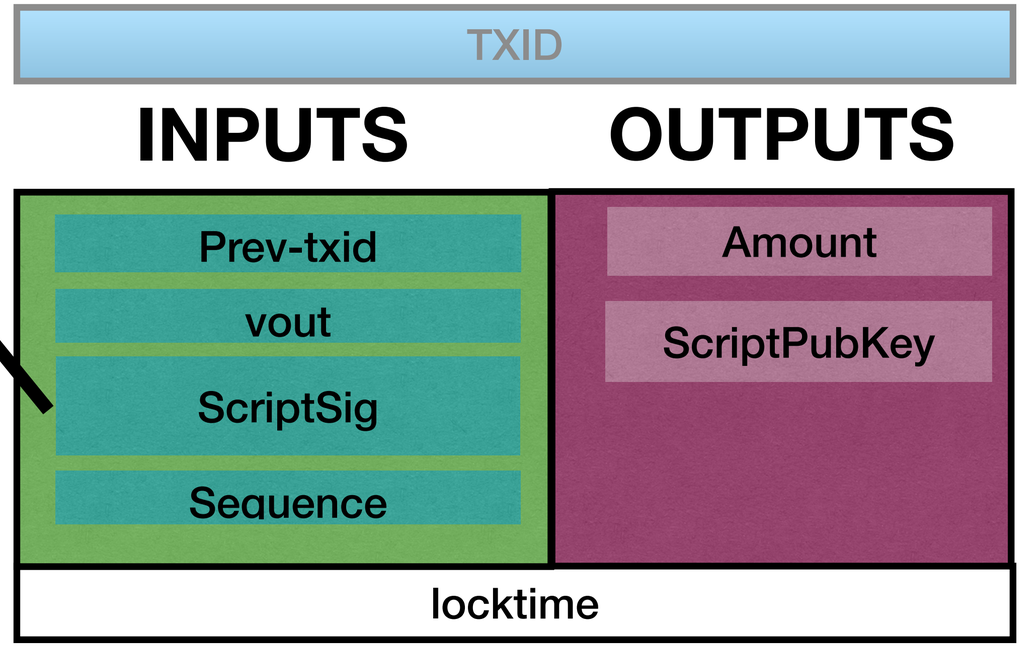
ScriptPubKey = locking

OP_DUP OP_HASH160 <pubkey_hash> OP_CHECKSIG

ScriptSig = unlocking

<signature> <pubkey>



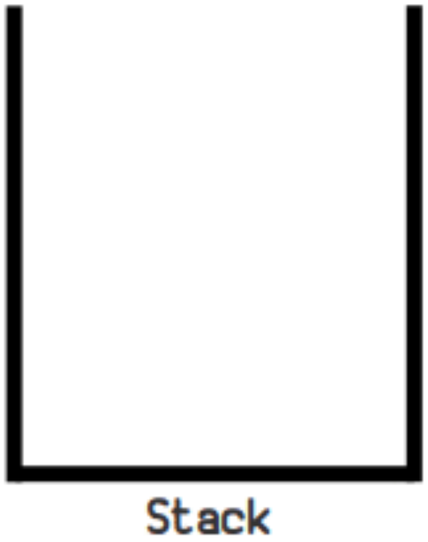


legacy P2PKH

Successful unlocking of previous utxo?

Standard Script: P2PKH





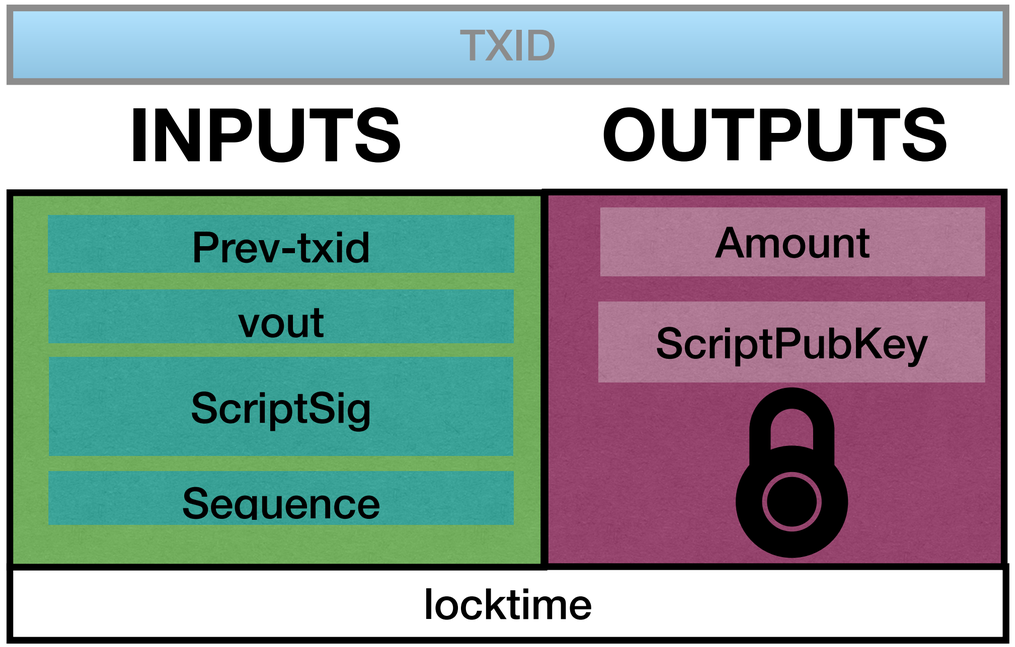
legacy P2SH

1. Legacy transaction

P2SH

legacy P2SH

P2SH locking script



ScriptPubKey

OP_HASH160 <script_hash> OP_EQUAL

legacy P2SH

P2SH locking script

ScriptPubKey

ASM | Hex

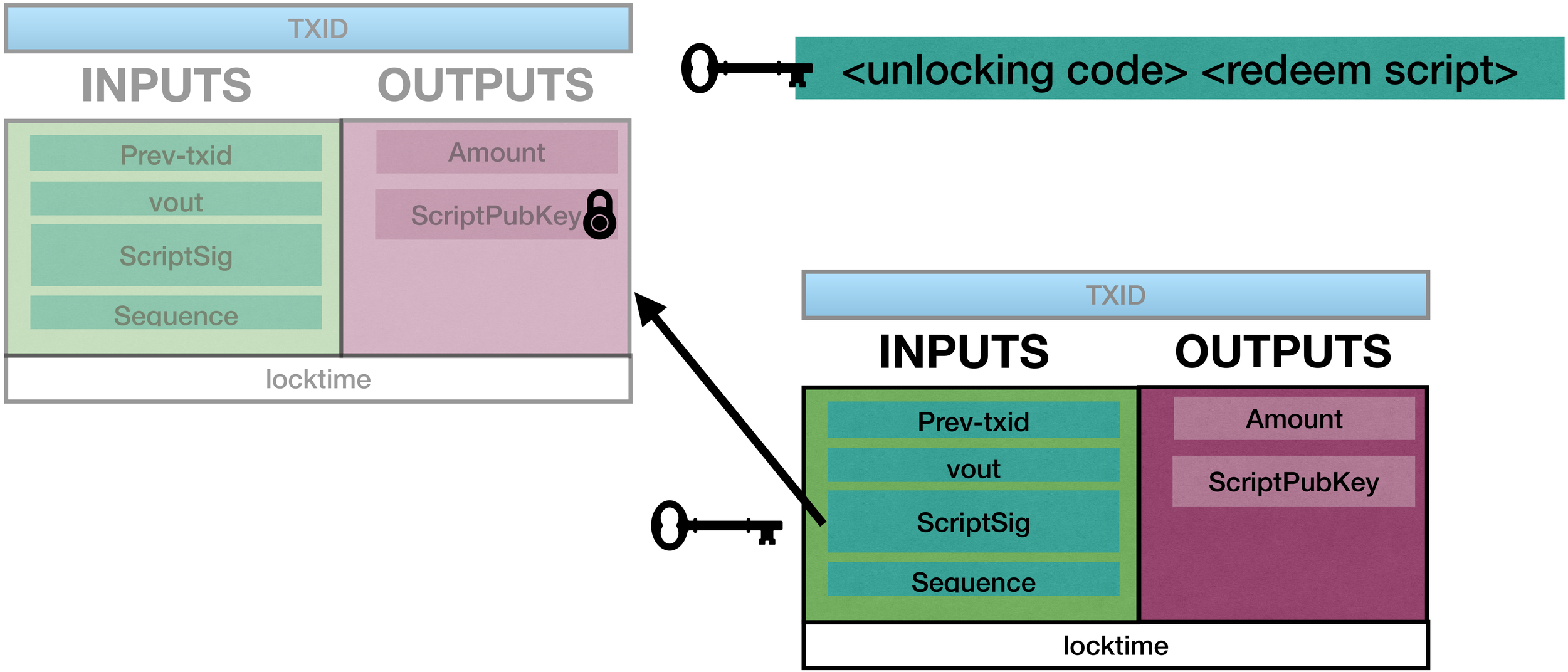
OP_HASH16
OP_PUSHBY
748284390f9e263a4b766a75d0633c50426eb875
OP_EQUAL

Script Hash (HASH160)

Transaction: 450c309b70fb3f71b63b10ce60af17499bd21b1db39aa47b19bf22166ee67144 (Output 1)

legacy P2SH

P2SH unlocking script

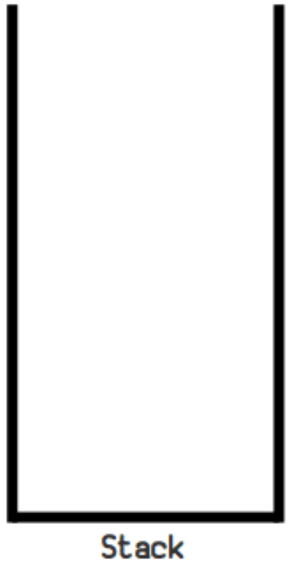


legacy P2SH

Successful unlocking of previous utxo?

Standard Script: P2SH





scriptPubKey

scriptSig

legacy

P2PK

<pubkey> OP_CHECKSIG

<signature>

P2PKH

OP_DUP OP_HASH160 <pubkey_hash>

<signature> <pubkey>

P2SH

OP_HASH160 <script_hash> OP_EQUAL

<unlocking code> <redeem script>

2. Segwit transaction

P2WPKH

P2WSH

Legacy transaction & Design goals



Privacy (don’t reveal too much user info)

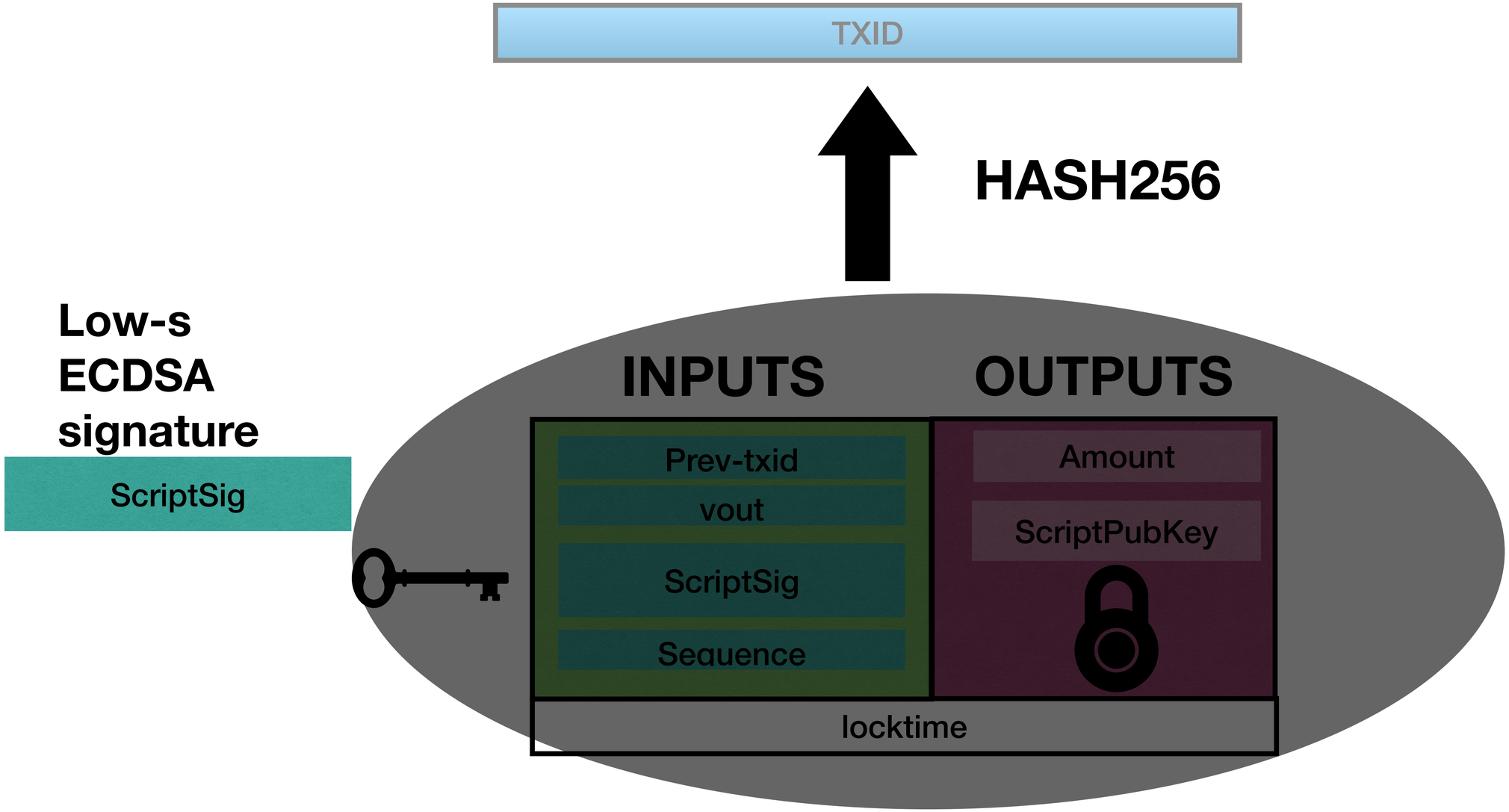
Scalability (block space is limited)

Immutability (no one tampers)

Security (only rightful owner spends)

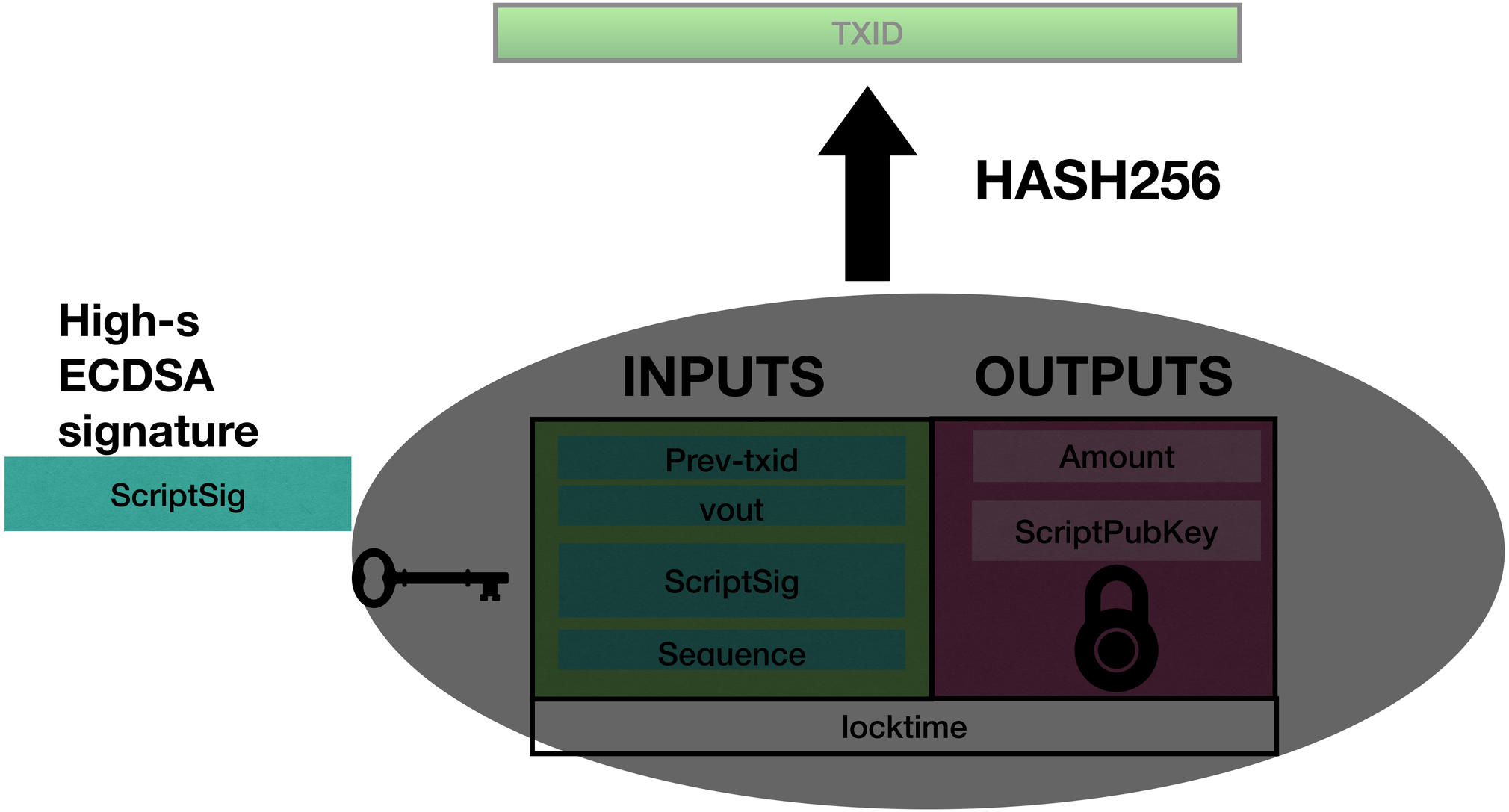
Legacy transactions & Immutability

TXID



Legacy transactions & Immutability

New TXID (everything else remains same)



Legacy transactions & Scalability

signature ~ 70% of the data weight of Block

More fees….

Legacy transactions & Privacy

Unlocking script/signature is exposed in full when someone “spends” bitcoin



scriptPubKey



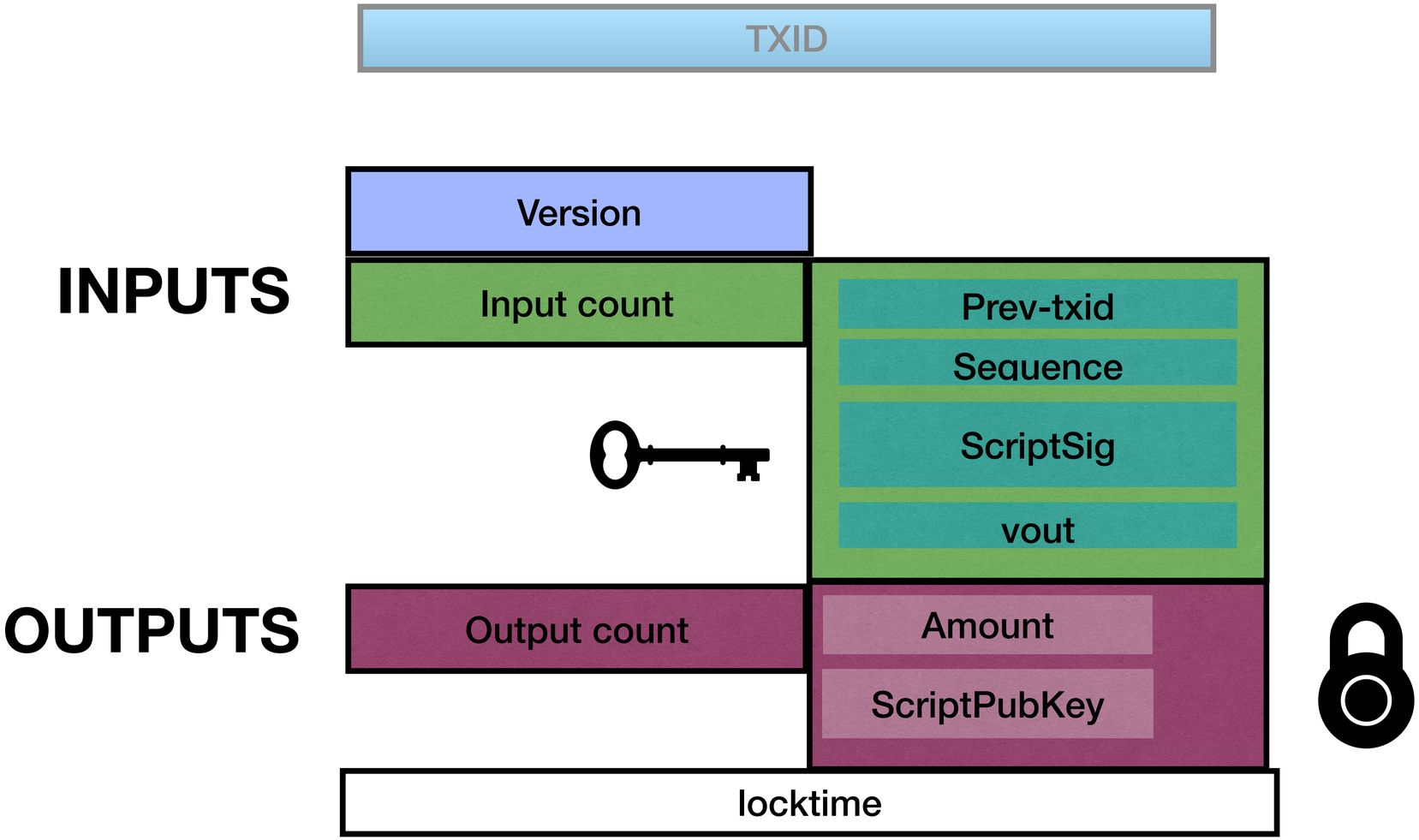
scriptSig

P2PKH

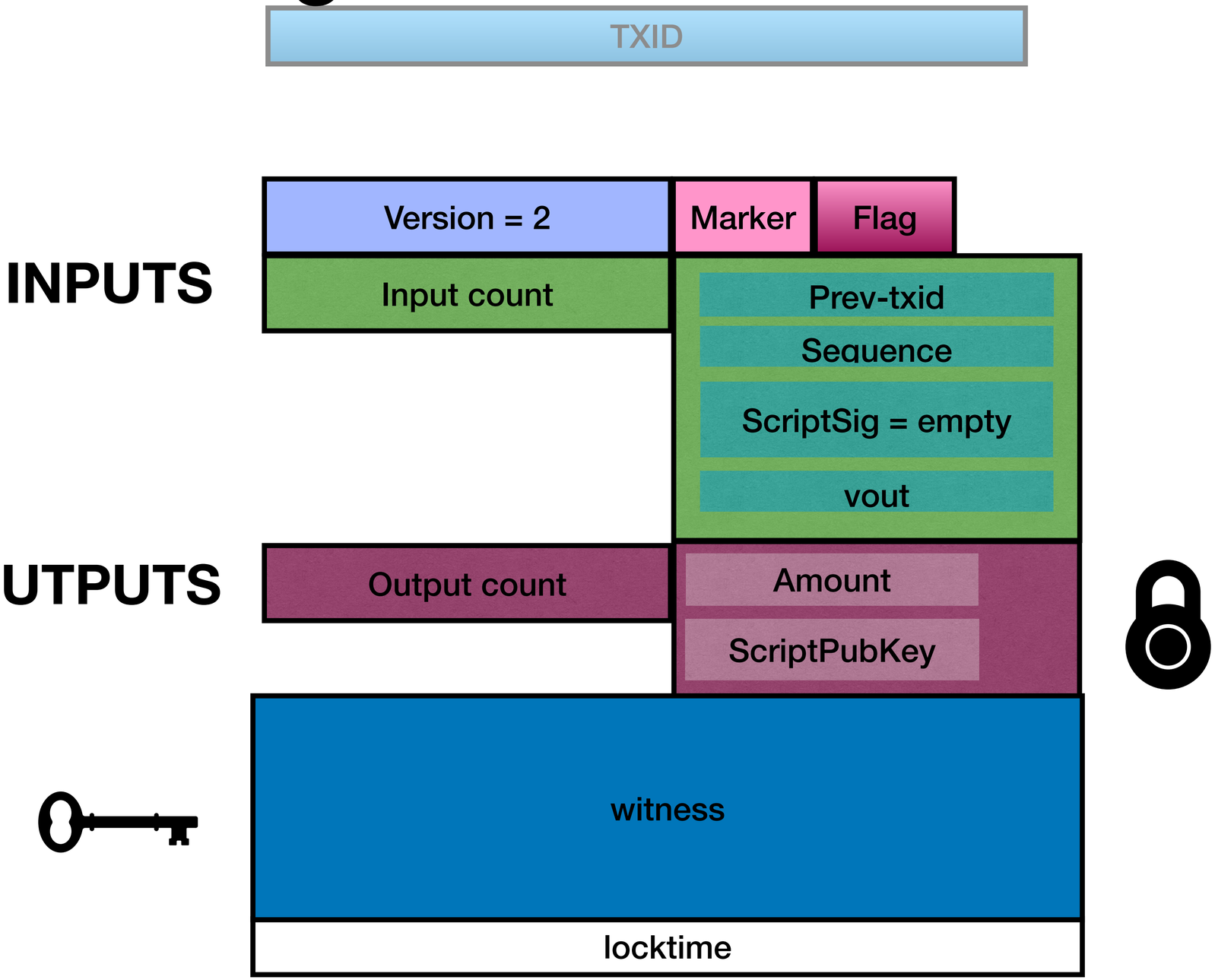
OP_DUP OP_HASH160 <pubkey_hash>

<signature> <pubkey>

Legacy transaction structure



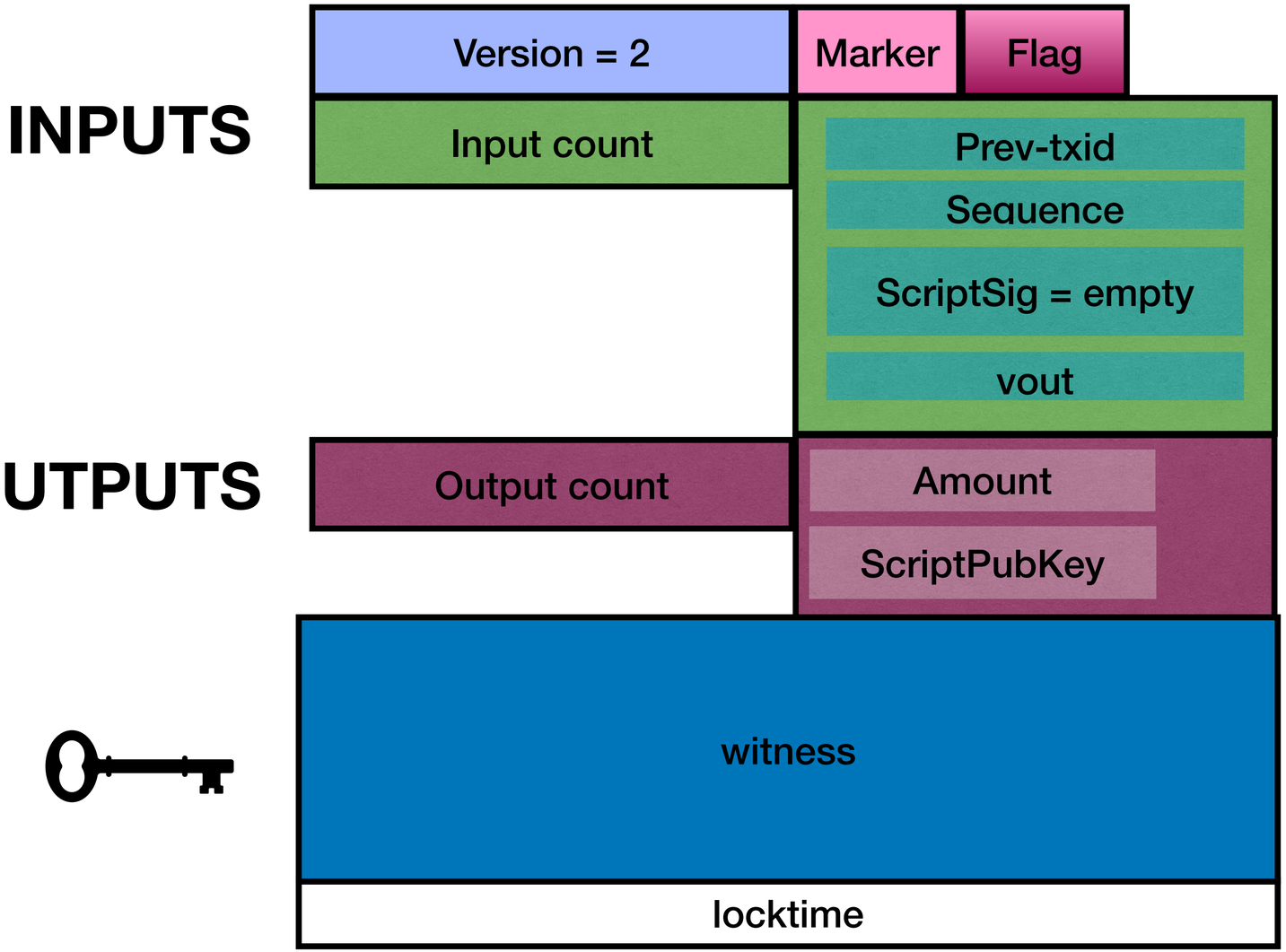
BIP141: Segwit introduced in 2017



segwit P2WPKH

P2WPKH

ScriptPubKey = locking script



ScriptPubKey

OP_0 <pubkey_hash>

segwit P2WPKH

P2WPKH locking script

ScriptPubKey

ASM | Hex

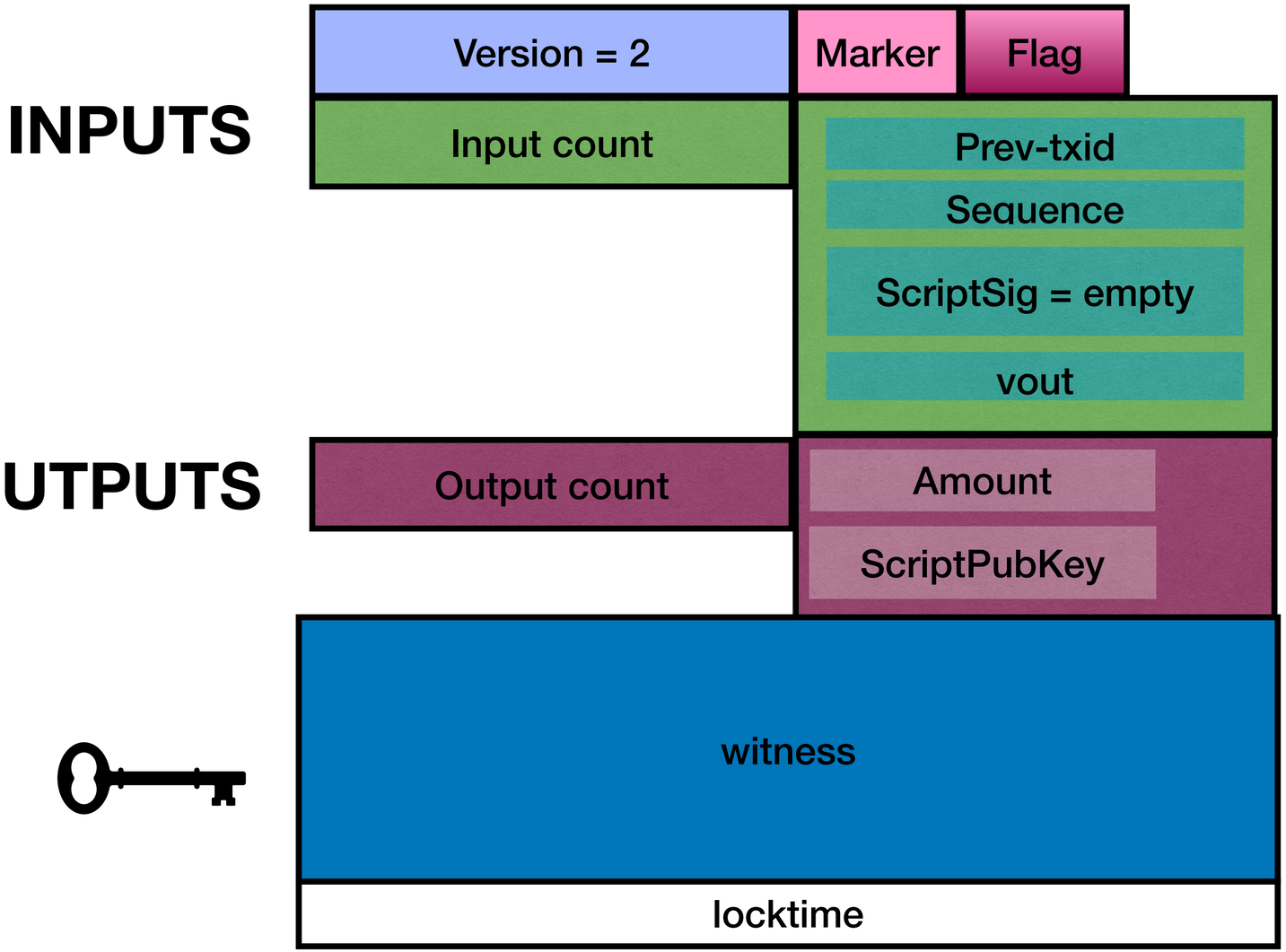
```
OP_0
OP_PUSHBYTES_20
841b80d2cc75f5345c482af96294d04fdd66b2b7
```

Transaction: c178d8dacdfb989f9d4fa45828ed188cd54a0414d625c3e61e75c5e3ac15a83a (Output 0)

segwit P2WPKH

P2WPKH

ScriptSig = empty



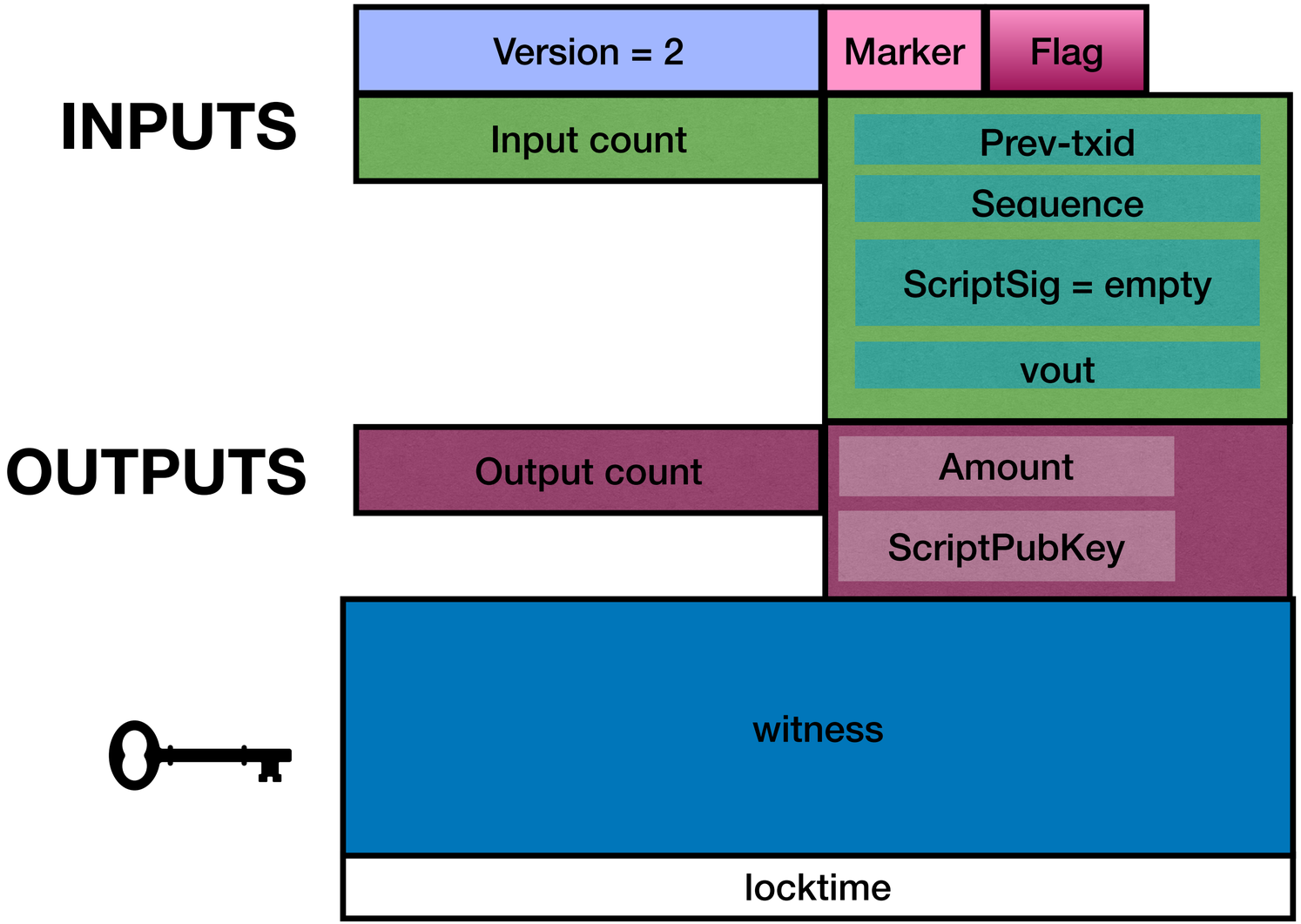
ScriptSig

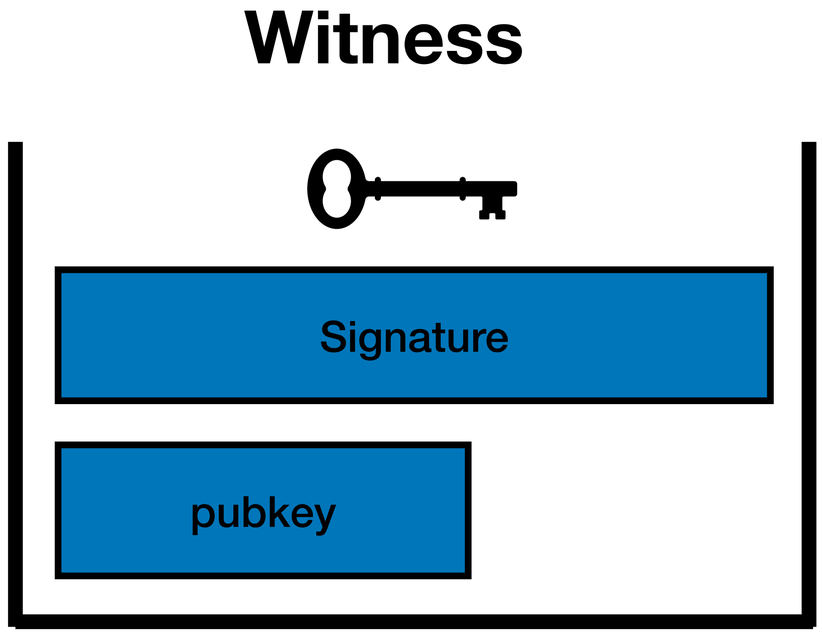
“”

segwit P2WPKH

P2WPKH

Witness = unlocking (not a script)



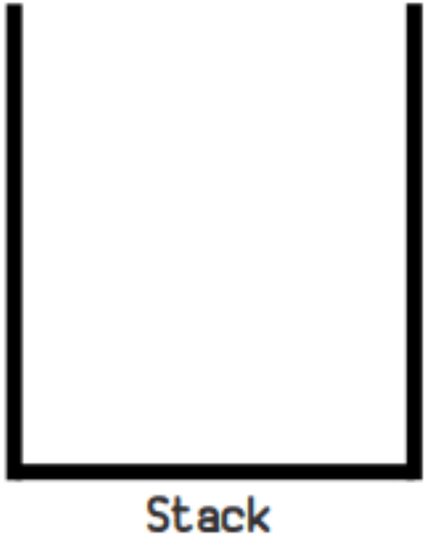


segwit P2WPKH

Successful unlocking of previous utxo?

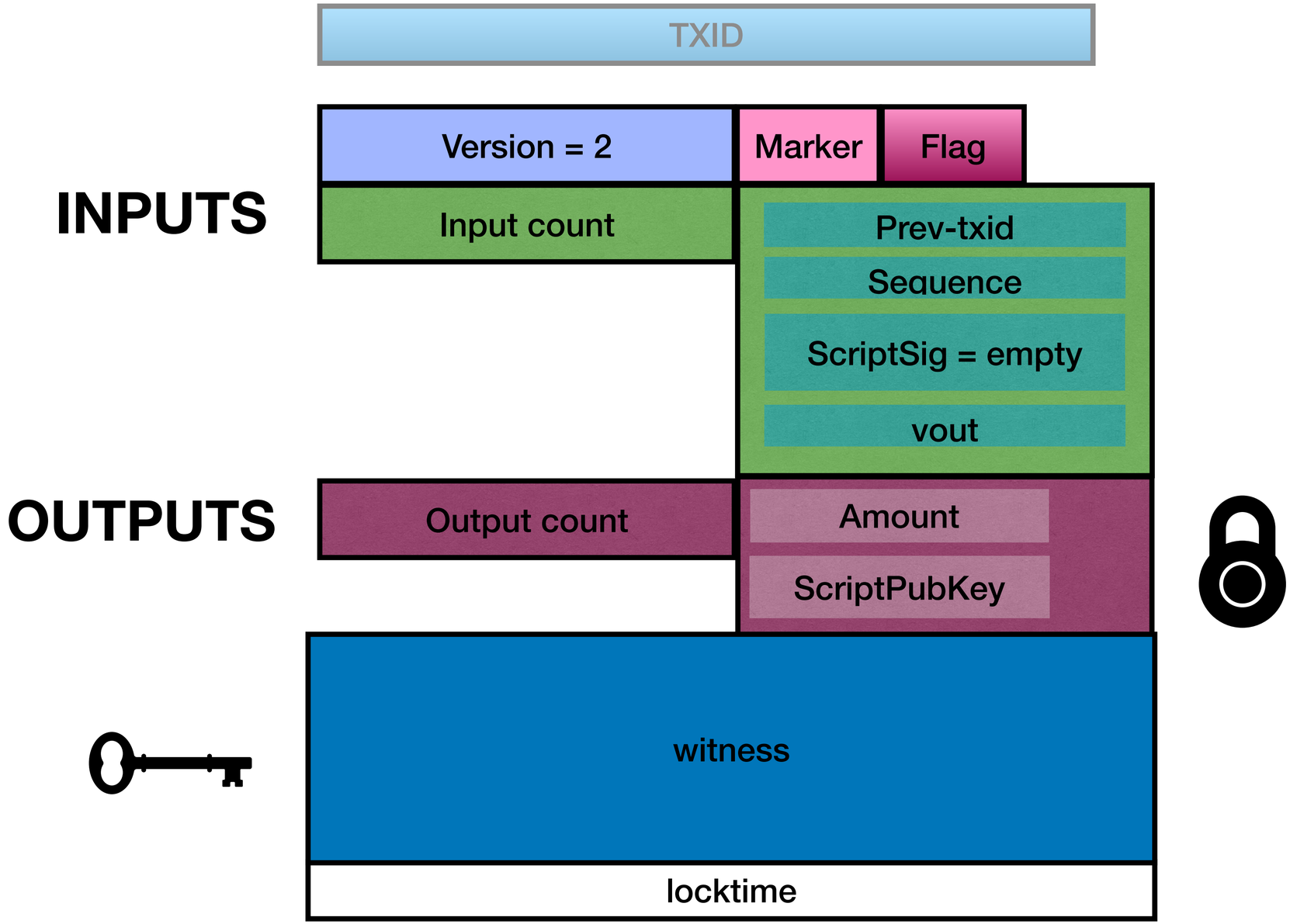
Standard Script: P2PKH





Legacy node encounters segwit transaction.

What Happens?



2. Taproot transaction

P2TR

Segwit transaction & Design goals



Privacy (don’t reveal too much user info)

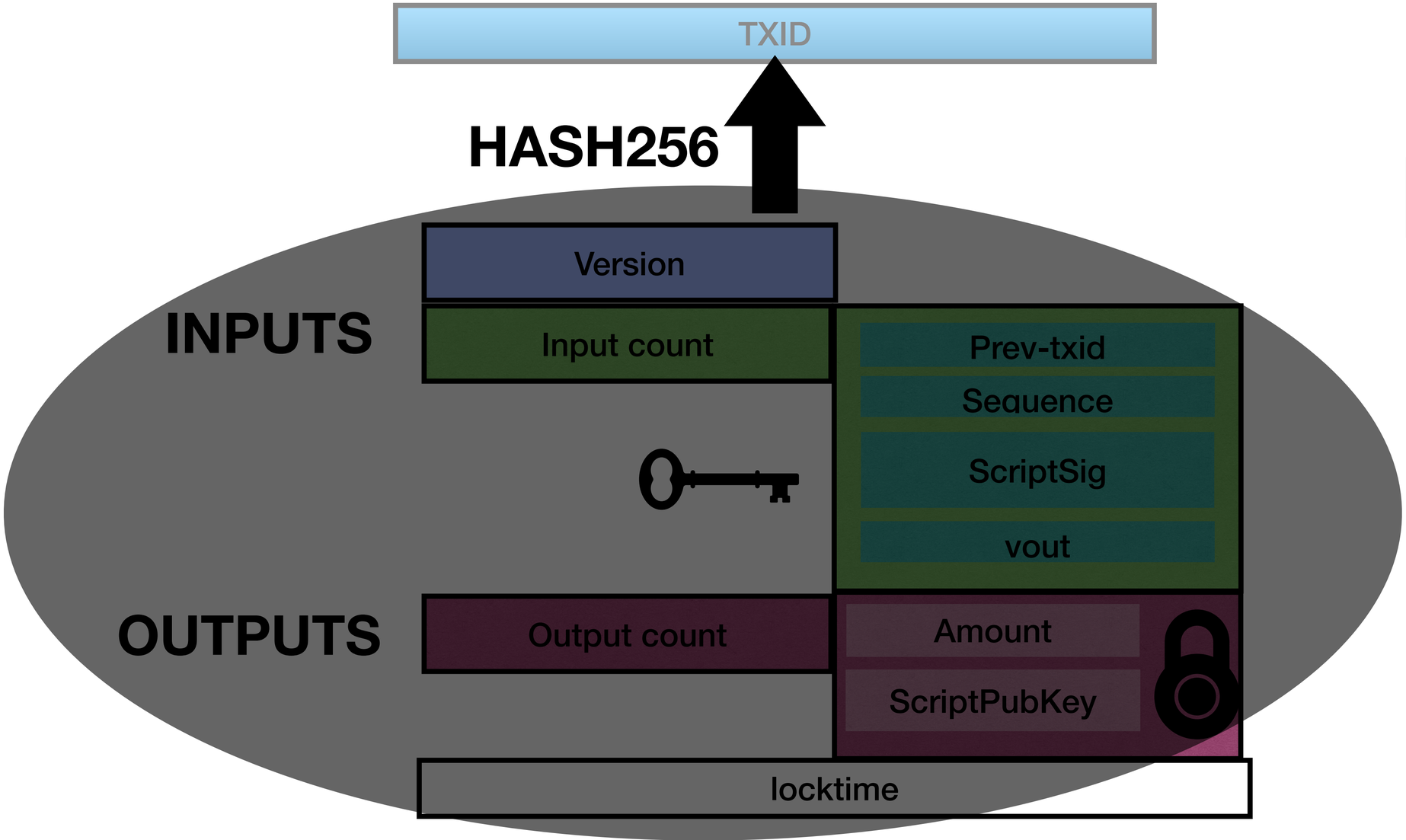
Scalability (block space is limited)

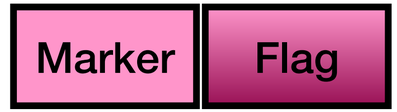
Immutability (no one tampers)

Security (only rightful owner spends)

Segwit transactions & Immutability

TXID remains same



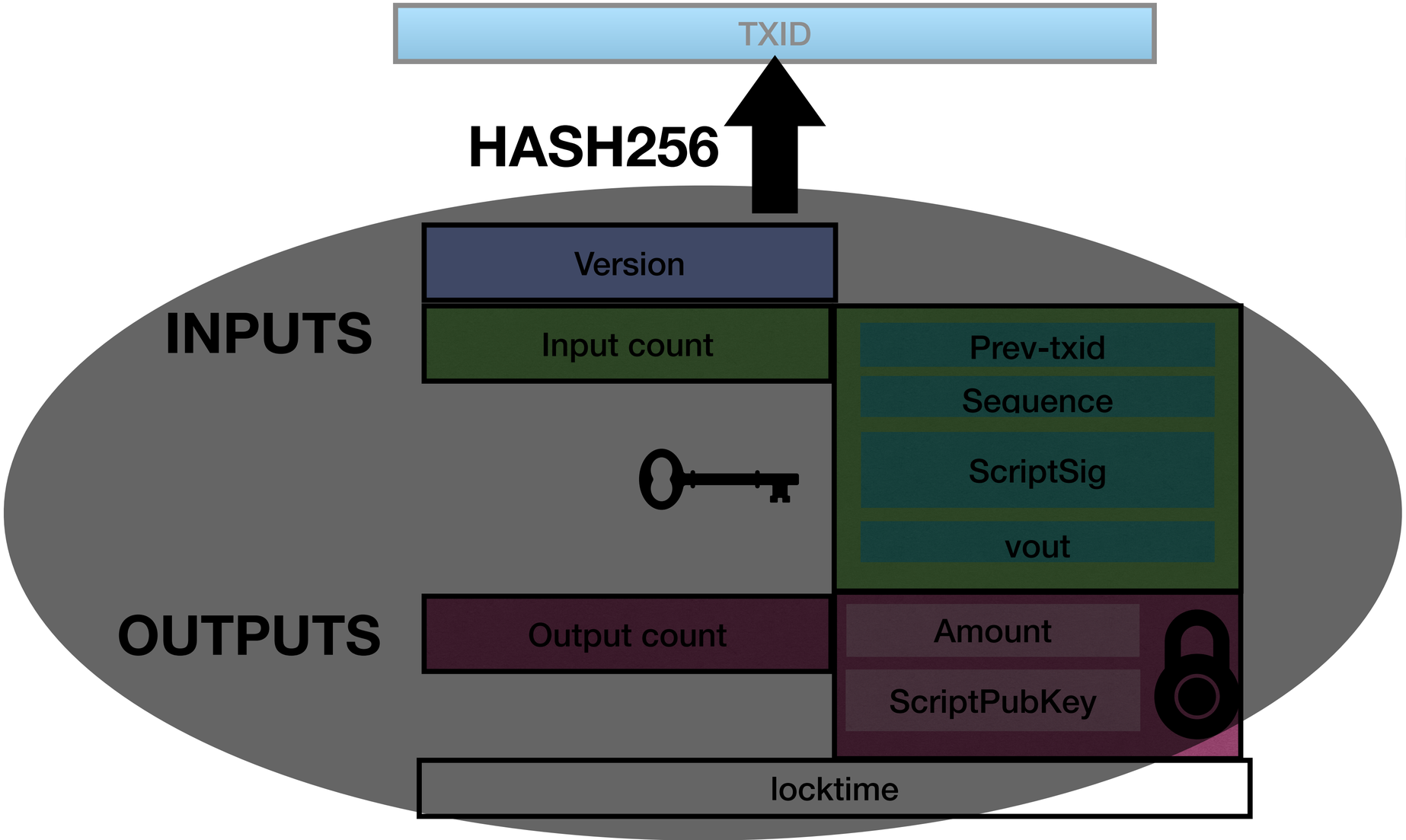


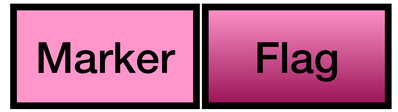
High-s ECDSA signature



Segwit transactions & Immutability

TXID



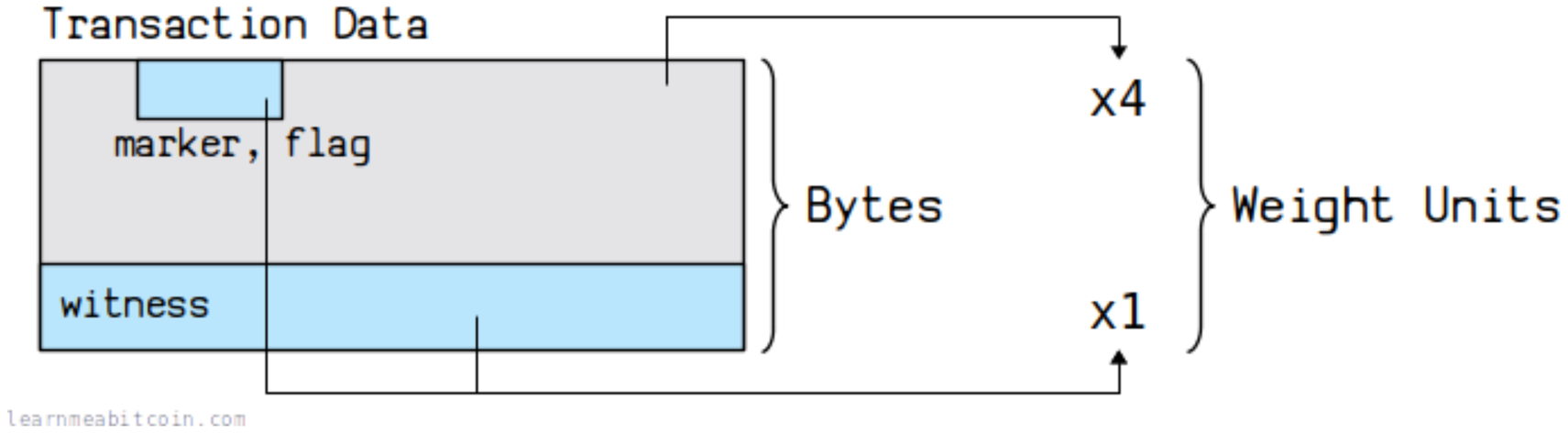


Low-s ECDSA signature



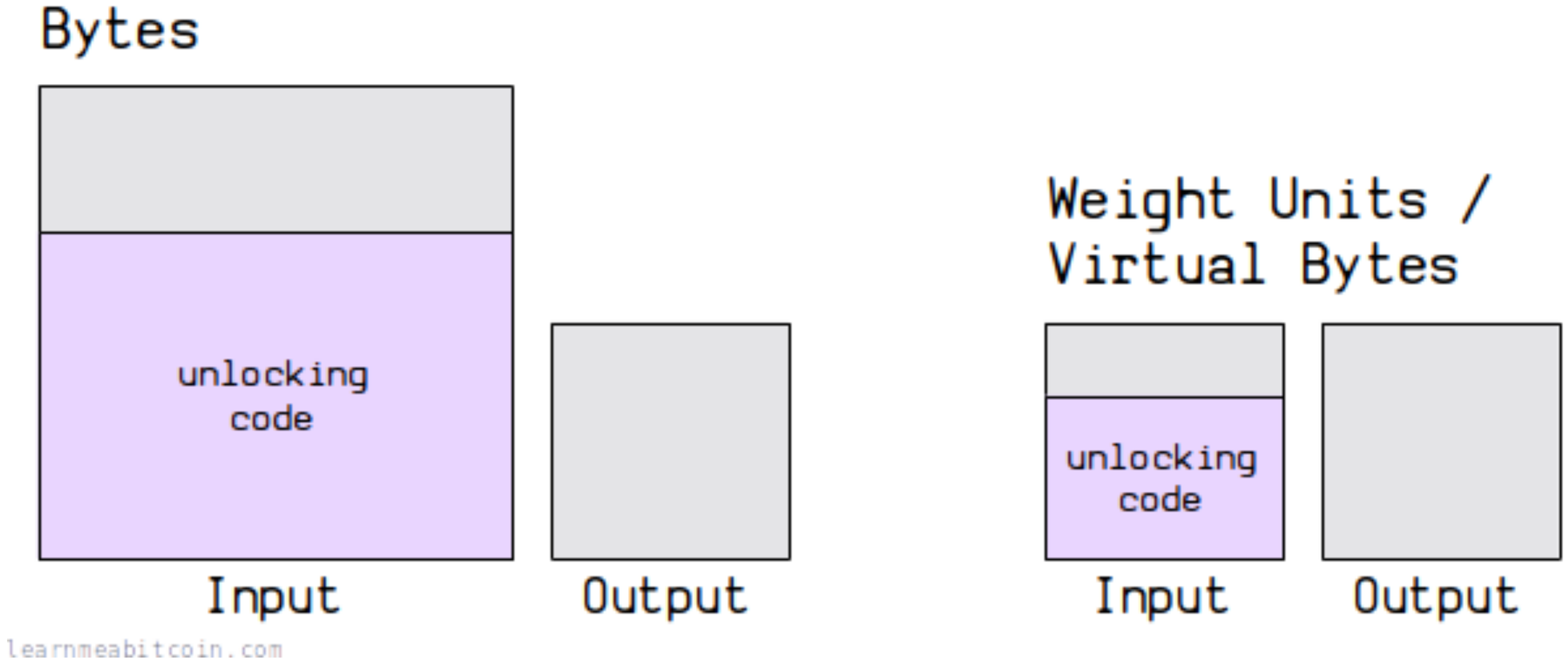
Segwit transactions & scalability

Measure transaction size with Weight Units = WU (instead of bytes)



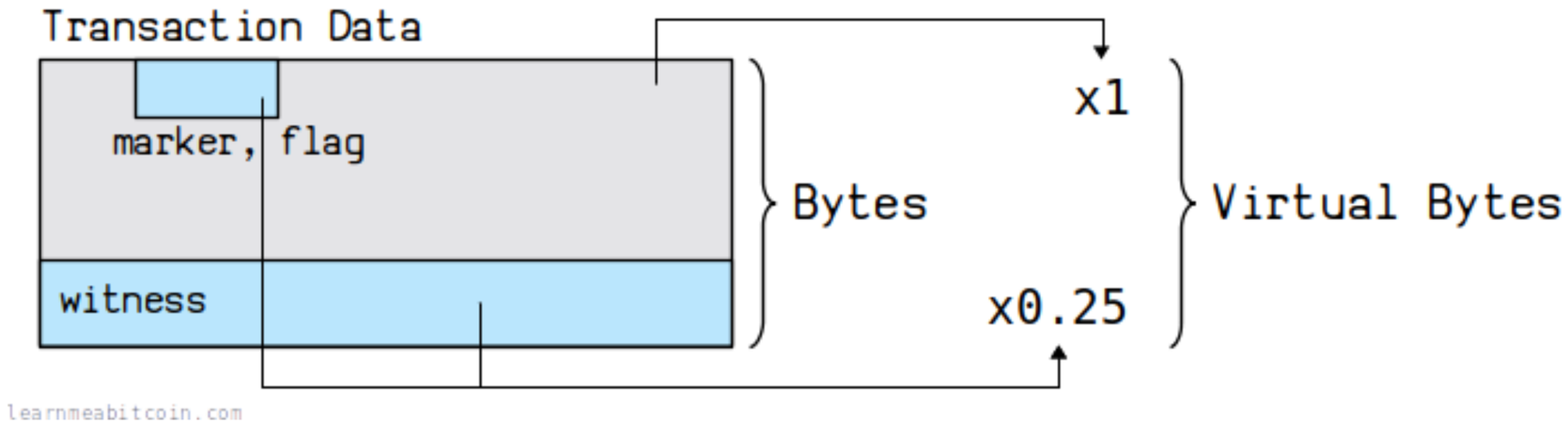
Segwit transactions & scalability

data required to unlock an output (i.e. signature data) is unfairly larger than the data required to put a lock on an output



Segwit transactions & scalability

Another measure of transaction size is virtual bytes (vB)
Ex: in mempool.space, fee rate = 1 sat/vB



Segwit transactions & Privacy

Unlocking script/signature is exposed in full when someone “spends” bitcoin



scriptPubKey

witness

P2WPKH

OP_0 <pubkey_hash>

<signature> <pubkey>

Segwit transactions & Privacy

“Current” policy is leaked on chain:

In 2-of-3 multisig

If X doesn’t respond within 30 days, let Y claim the funds.



scriptPubKey

witness

P2WSH

OP_0 <script_hash>

<signature> <signature> <script>

Segwit transactions & Privacy

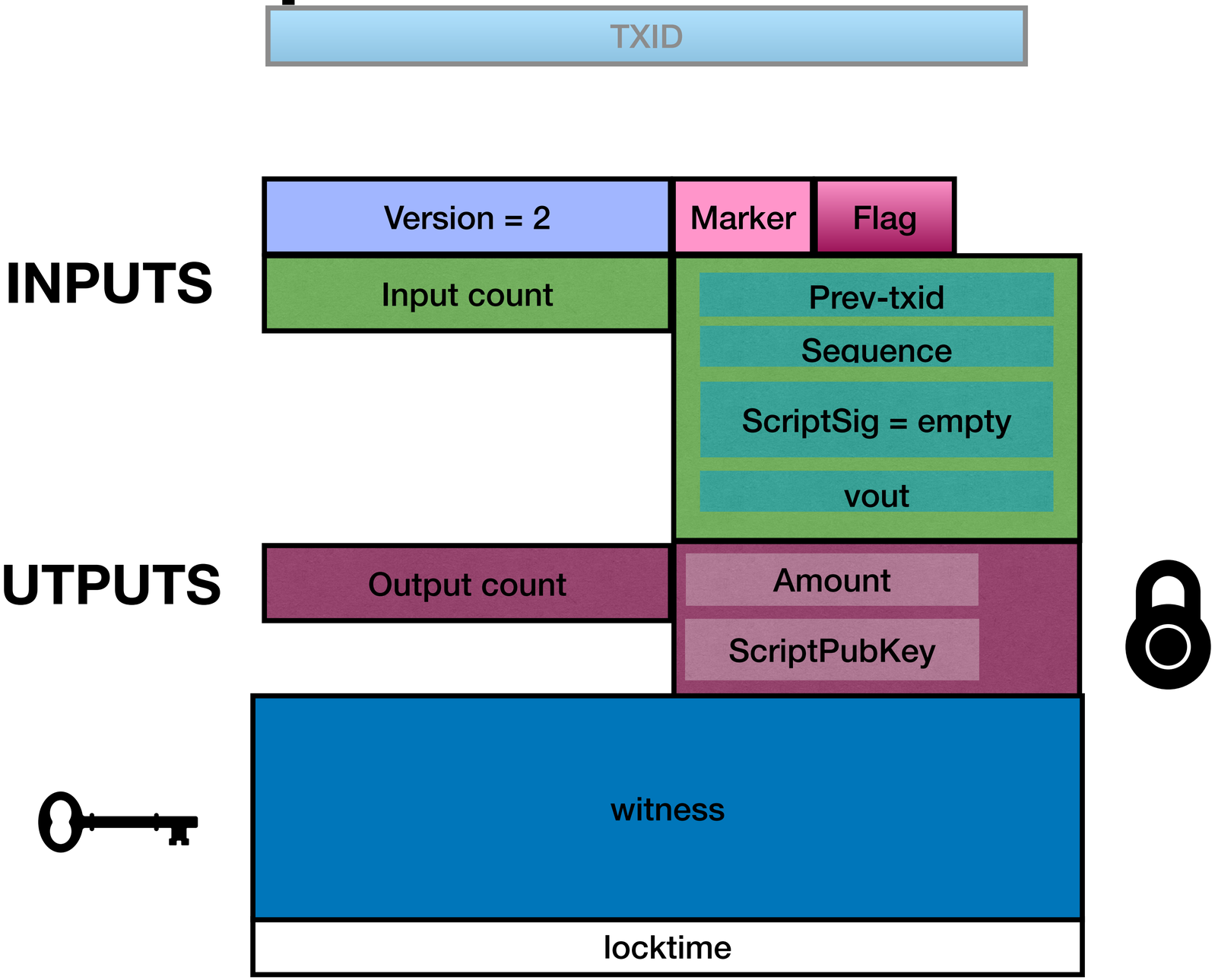
“Multiple” exact policy is leaked on chain:

A business wants to secure its Bitcoin treasury using the following logic:

Spending Conditions:

1. **Primary Path (Day-to-Day Use):**
   - Spend with a **2-of-3 multisig** (e.g., CEO + CFO + Legal)
2. **Backup Path (Disaster Recovery):**
   - After 30 days, allow a **single emergency key** to spend the funds.

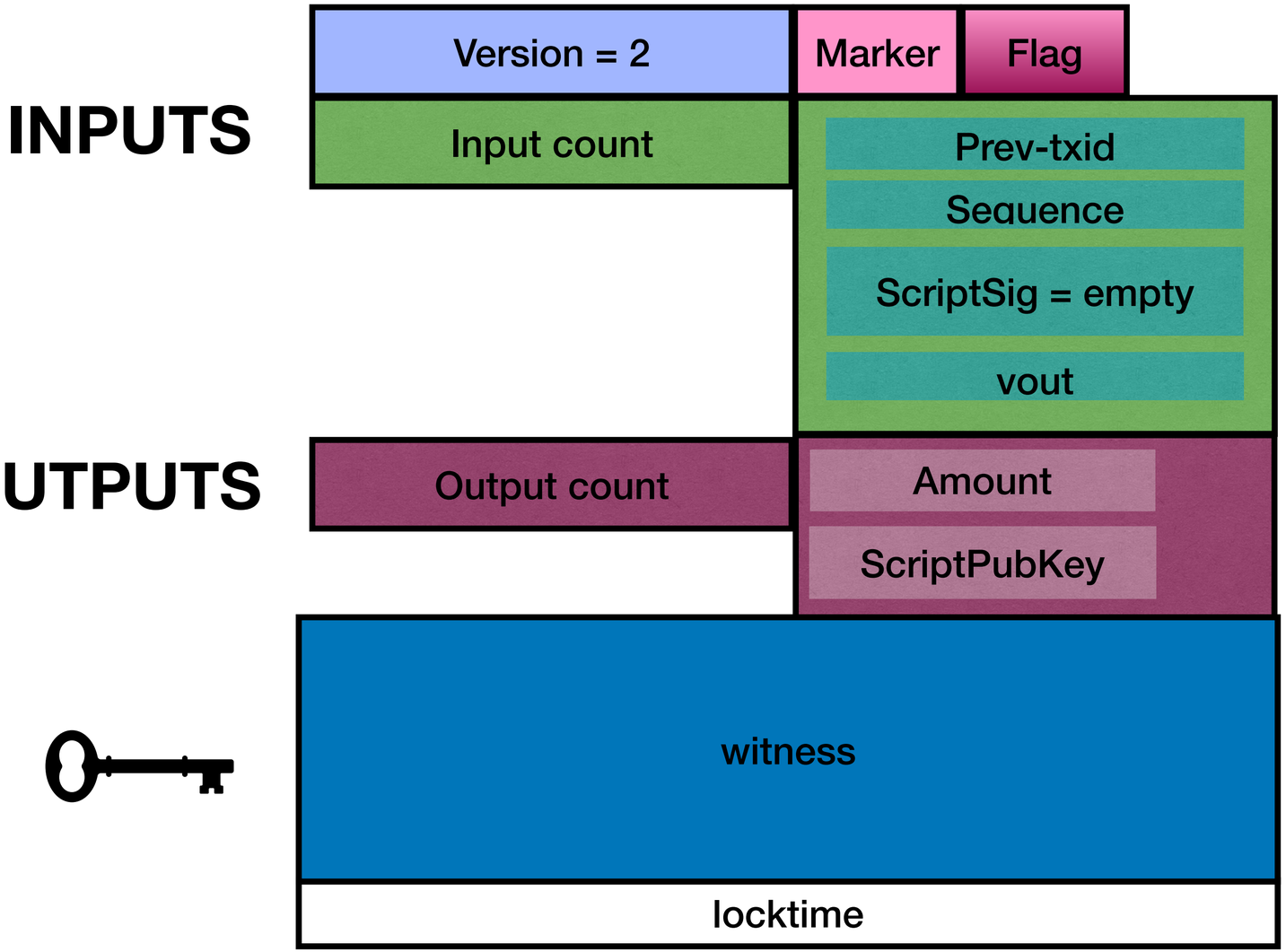
BIP341: Taproot introduced in 2021



taproot P2TR

P2TR

ScriptPubKey = locking script



ScriptPubKey

OP_1 <32 byte tweakpubkey>

taproot P2TR

P2TR locking script

ScriptPubKey

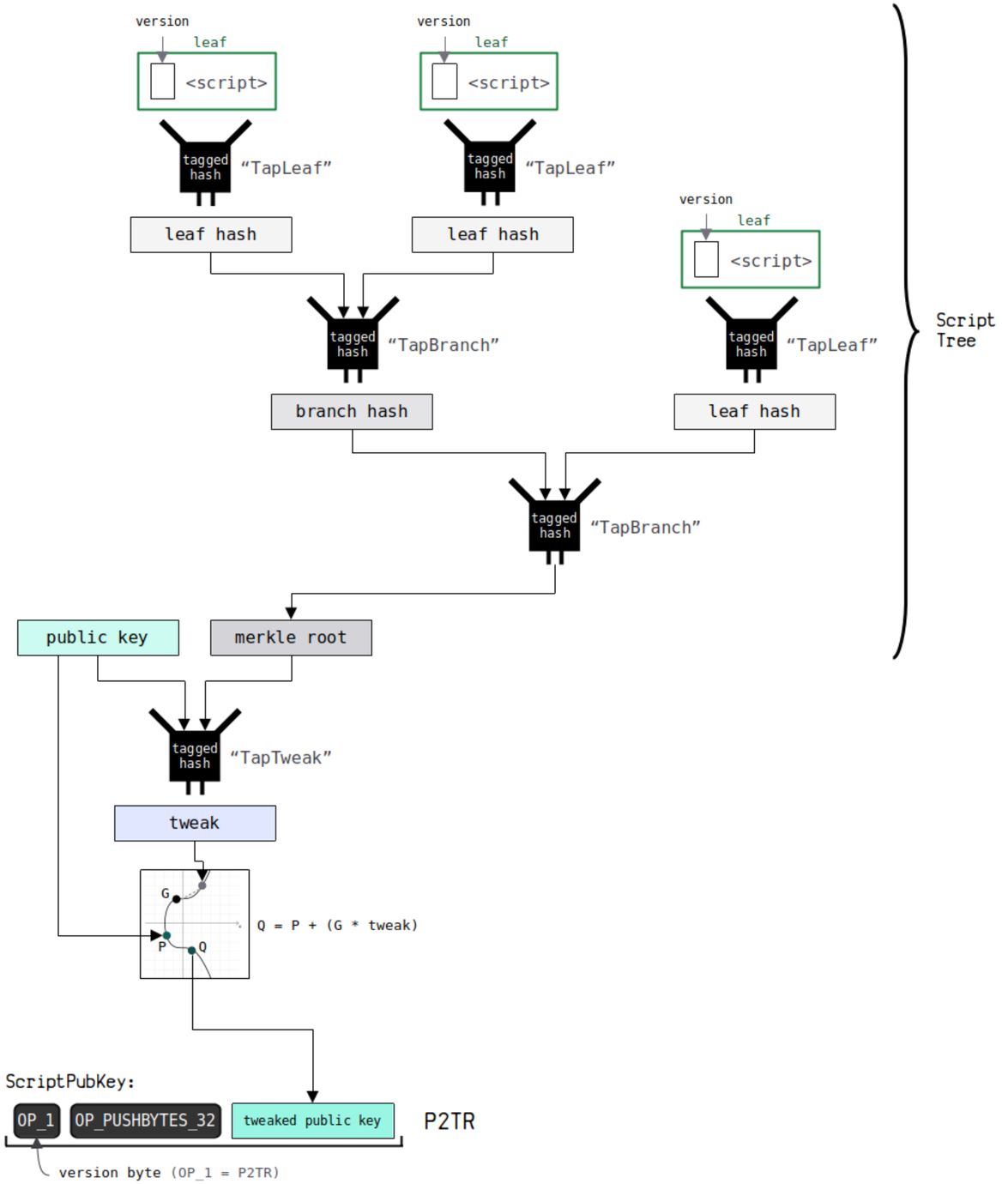
ASM Hex

OP_1
OP_PUSHBYTES_32
562529047f476b9a833a5a780a75845ec32980330d76d1ac9f351dc76bce5d72

Tweaked Public Key

Transaction: 0c27dceb3c5e8aeffaad5c01229eac985b4a3d56388fcef07daecb1b5482495e (Output 0) (Used for key path spend ↘)

Transaction: 88047644c7e42421861b5d15551aa29151f86d81409fd9a3831f43a541505720 (Output 1) (Used for script path spend ↘)



learnmeabitcoin.com

P2TR unlocking script

P2TR “locking script” allows you to unlock output using choice of custom script

Key path spend

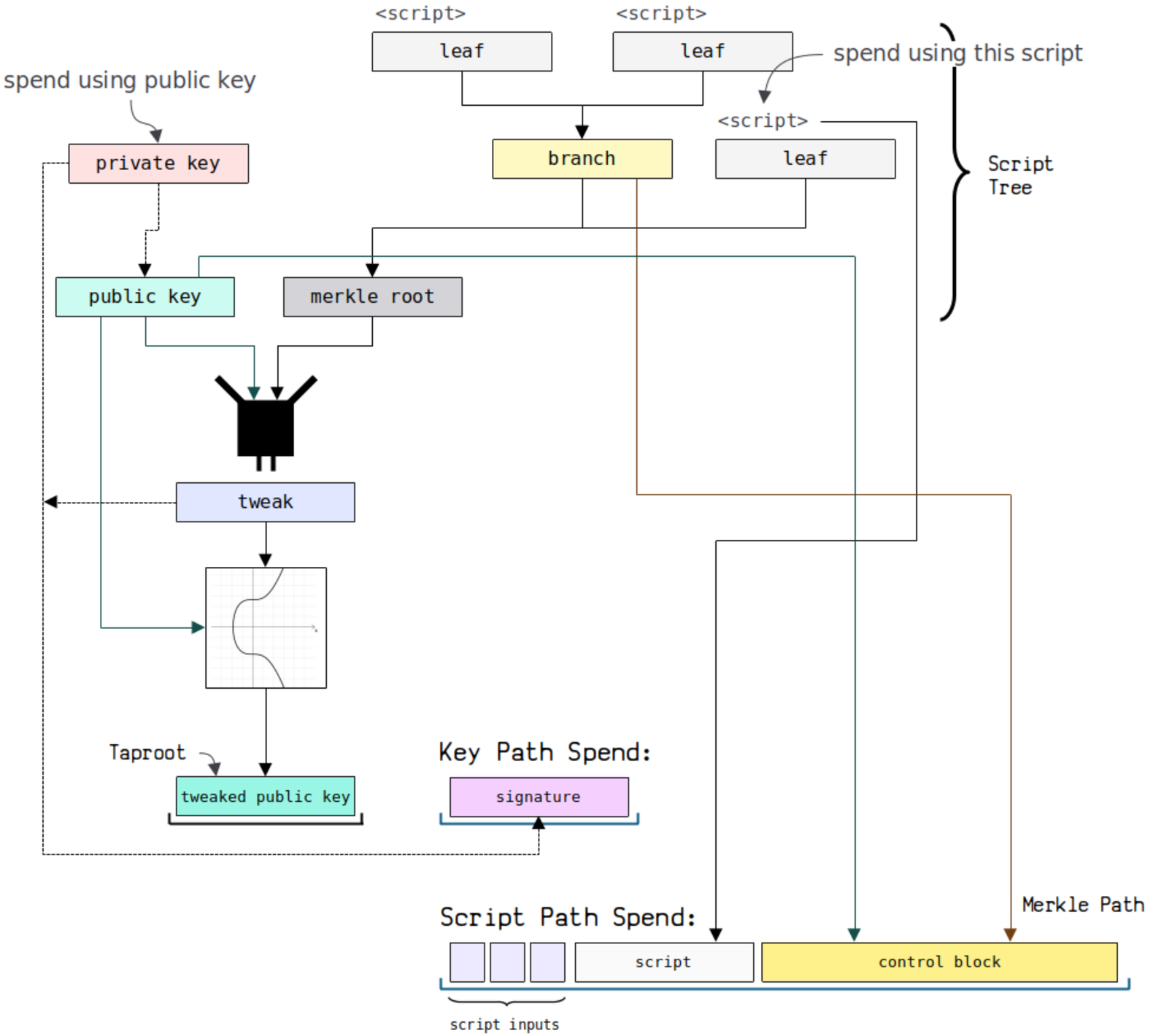
Similar to P2WPKH

Signature

Script path spend

Similar to P2WSH

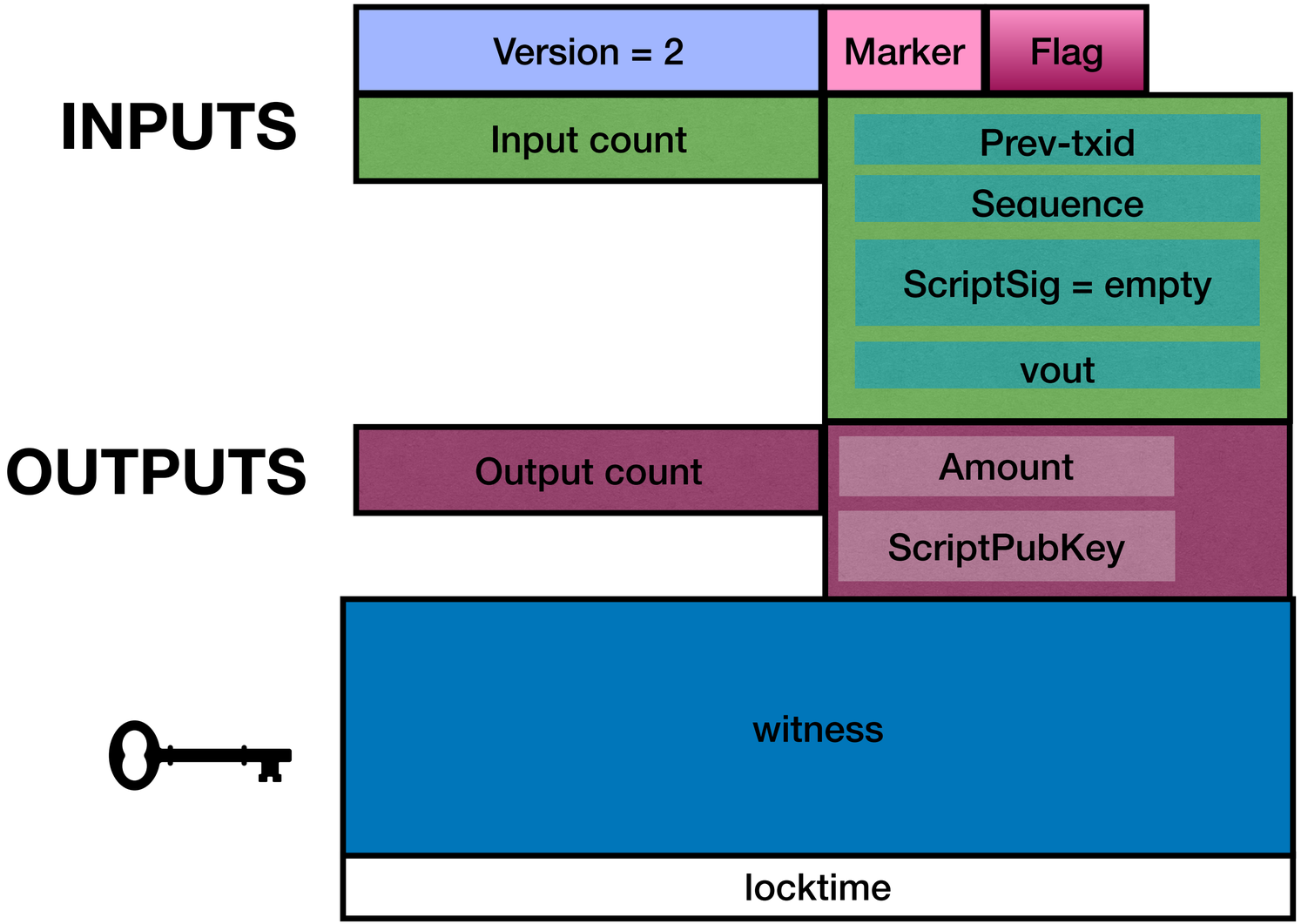
1 of multiple custom scripts to unlock from


<script>
<script>
leaf
leaf
spend using this script
spend using public key
<script>
private key
branch
leaf
Script Tree
public key
merkle root
tweak
Taproot
tweaked public key
Key Path Spend:
signature
Merkle Path
Script Path Spend:
script
control block
script inputs

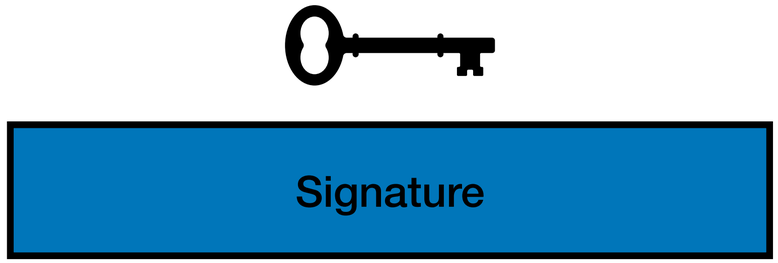
taproot P2TR

P2TR

Witness = unlocking (not a script)



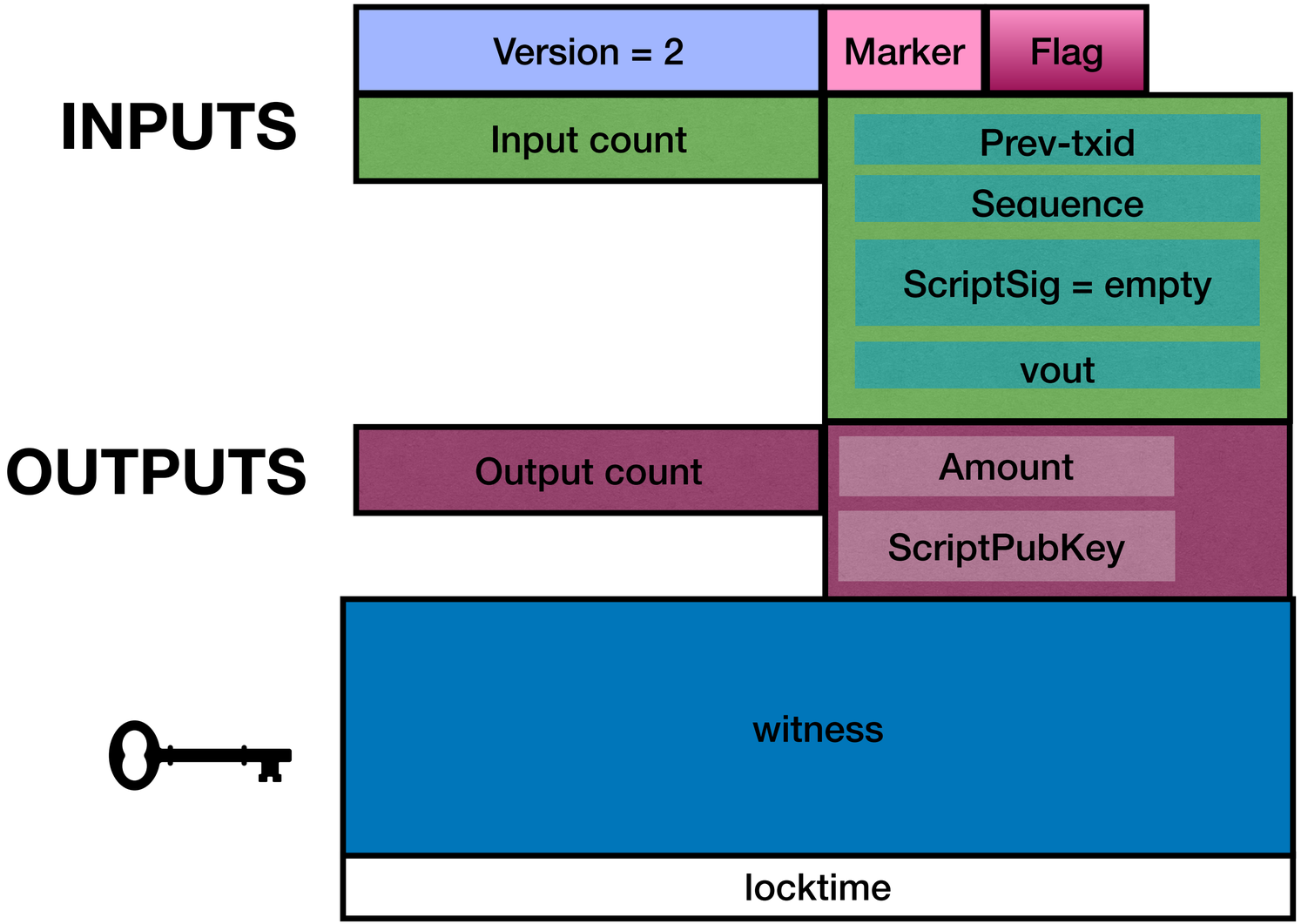
Key path spend witness



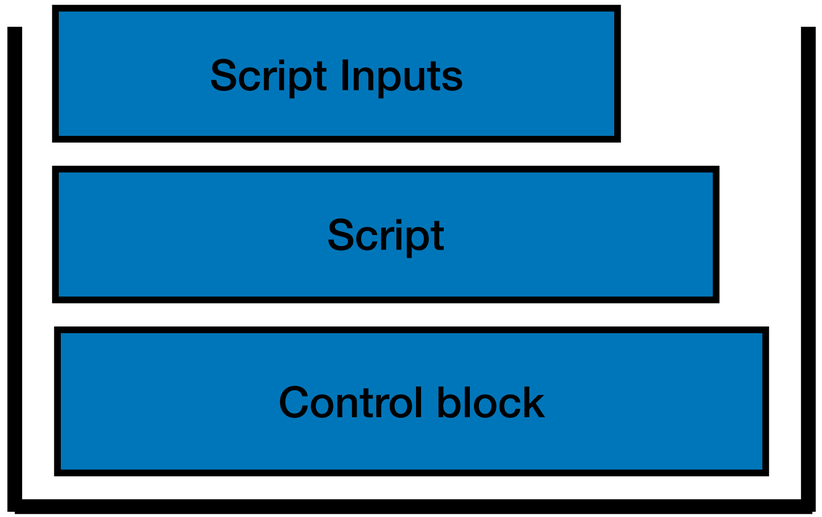
taproot P2TR

P2TR

Witness = unlocking (not a script)



Script path spend witness



Taproot transactions & Design goals



Privacy (don’t reveal too much user info)

Scalability (block space is limited)

Immutability (no one tampers)

Security (only rightful owner spends)

Taproot transactions & Privacy

“Current” policy is leaked on chain:

In 2-of-3 multisig

If X doesn’t respond within 30 days, let Y claim the funds.



scriptPubKey

witness

P2WSH

OP_1 <tweaked pubkey>

<inputs> <script> <control block>

Taproot transactions & Privacy

“Multiple” exact policy is NOT leaked on chain:

A business wants to secure its Bitcoin treasury using the following logic:

Spending Conditions:

1. **Primary Path (Day-to-Day Use):**
   - Spend with a **2-of-3 multisig** (e.g., CEO + CFO + Legal)
2. **Backup Path (Disaster Recovery):**
   - After 30 days, allow a **single emergency key** to spend the funds.

Taproot transactions & scalability

Schnorr signatures allow combining multiple signatures into 1 signature

Multiple spending conditions -> 32 byte Merkle root

Conclusion

transaction design goals



Taproot

Privacy (don’t reveal too much user info)

Taproot Segwit

Scalability (block space is limited)

Taproot Segwit

Immutability (no one tampers)

Taproot Segwit Legacy

Security (only rightful owner spends)

Next time you make a transaction, Check what kind of transaction you’re making!



menti.com

enter code: 6975 5565

